

Final REPORT

Big Lab B

汇报人：张硕

2026.05.30



Chapter

chapter

Part 01

BigLab A Review



简要回顾

完成了若干个基础实验

支持/开发了若干个游戏



Chapter

Chapter

Part 02

—

BigLab B T1

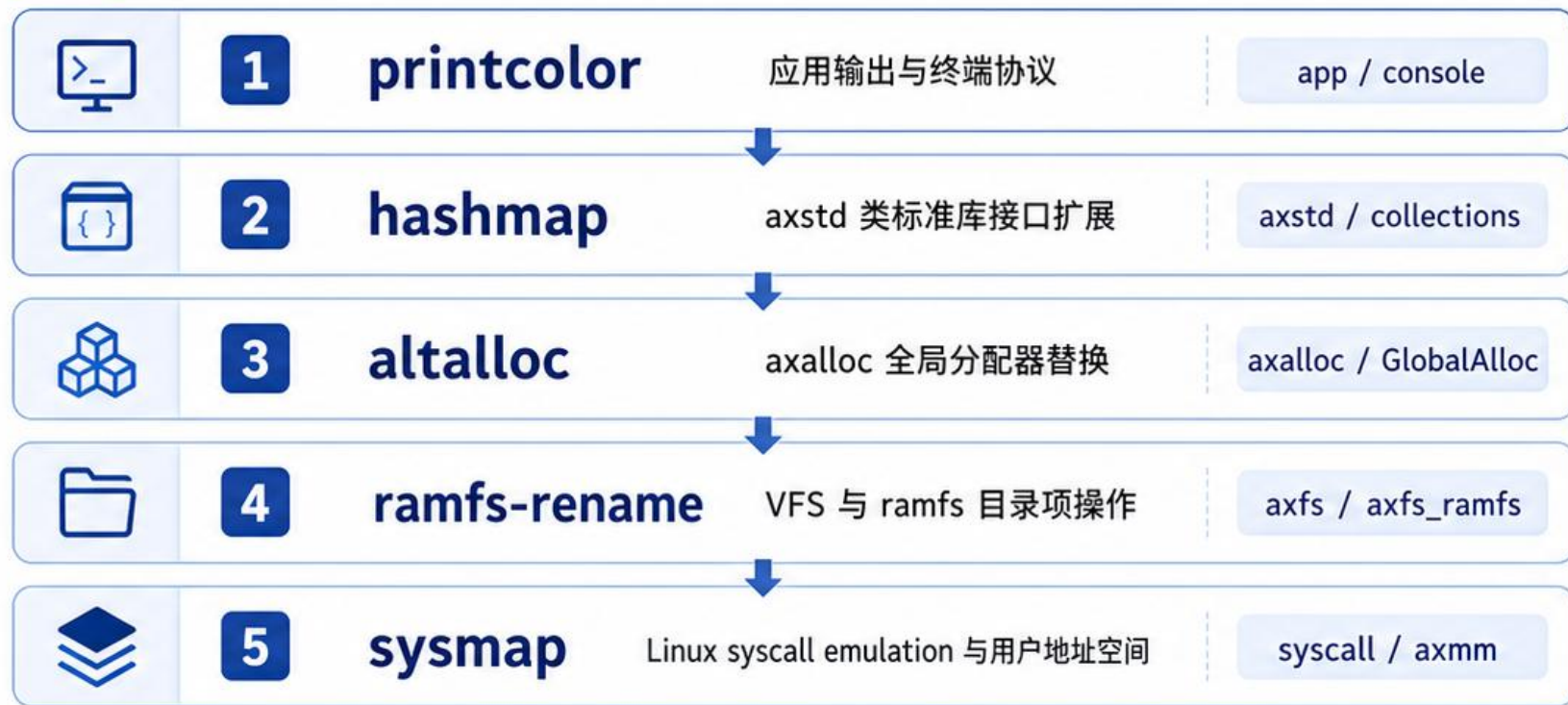


简要回顾

- printcolor
 - hashmap
 - altalloc
 - ramfs-rename
 - sysmap
- 

从应用输出到系统调用的逐层深入

五个基础 exercise 覆盖 ArceOS 的关键技术层次



贯穿主线



从应用 API 观察行为



沿 axstd / alloc / fs 下探实现



关注状态变化与边界条件



通过测试输出闭环验证



验证闭环：测试输出驱动逐层验证



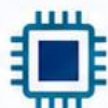
输出颜色

终端颜色与格式输出



HashMap 行为

插入 / 查询 / 冲突



分配器切换

分配 / 释放 / OOM



rename 结果

目录项重命名验证



mmap 路径

映射 / 读写 / 结果



Chapter

Chapter

Part 02

—

BigLab B T1

starry-evolve: AI 驱动的 StarryOS 内核持续改进框架



面向 **syscall** 兼容性改进



AI-coding agent 负责任务推进



确定性工具负责验真



Linux-vs-StarryOS 对拍作为核心验证路径



以 **AI** 推进问题修复，以确定性验证守住可信边界

解决什么问题?

StarryOS 是一个教学/研究用途的 OS 内核，其 syscall 实现存在三类问题：

1. **未实现 (STUB)**：dispatch 表中有条目，handler 为 `todo!()` 或空函数，运行时会 panic。
2. **语义偏差 (SEMANTIC_MISMATCH)**：函数存在，但返回值、errno、参数验证顺序等行为与 Linux 不一致。
3. **部分实现 (PARTIAL)**：覆盖了主路径，但缺少边界情况处理。

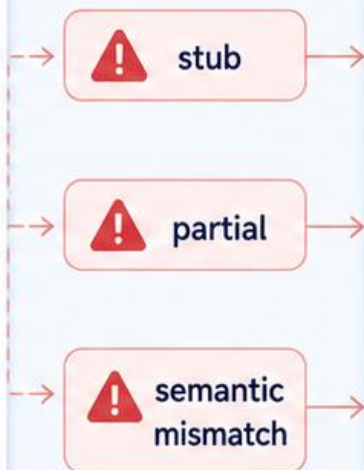
手工修复这些问题需要反复查阅 Linux man page、编写测试点、在 QEMU 中运行验证——周期长且容易遗漏。starry-evolve 将这个流程结构化为一组确定性工具和 AI 辅助技能，使得改进过程可复现、可审查。

问题背景：为什么内核改进需要框架化

手工修复链路



🕒 周期长，反馈慢



框架化闭环



✅ 可复现，可追踪



StarryOS 需要对齐 Linux syscall 行为



实现中可能存在 stub、partial 和语义偏差

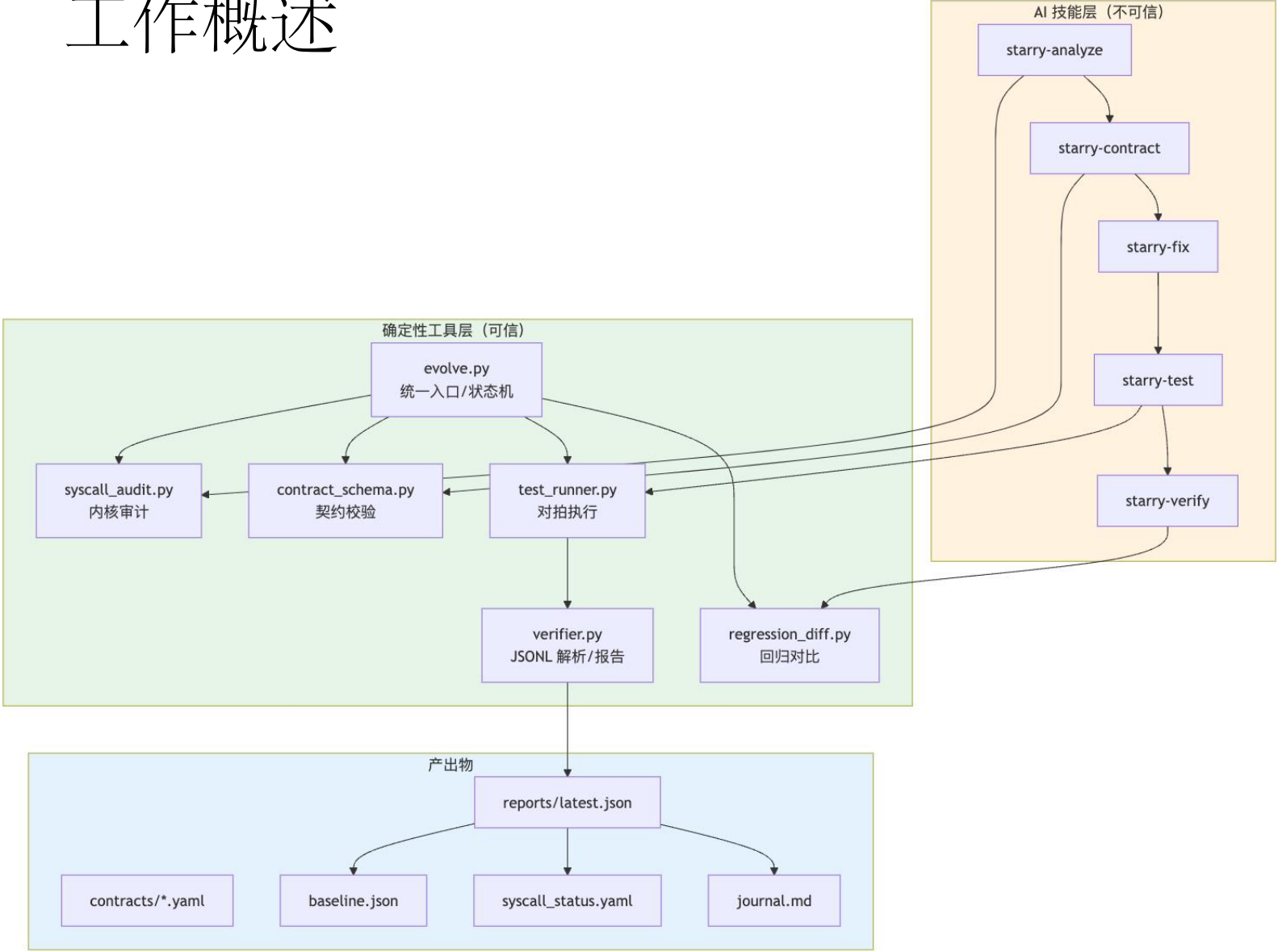


手工查规范、改内核、写测试、跑 QEMU 周期长



AI 可以提速，但不能直接充当最终判定者

工作概述



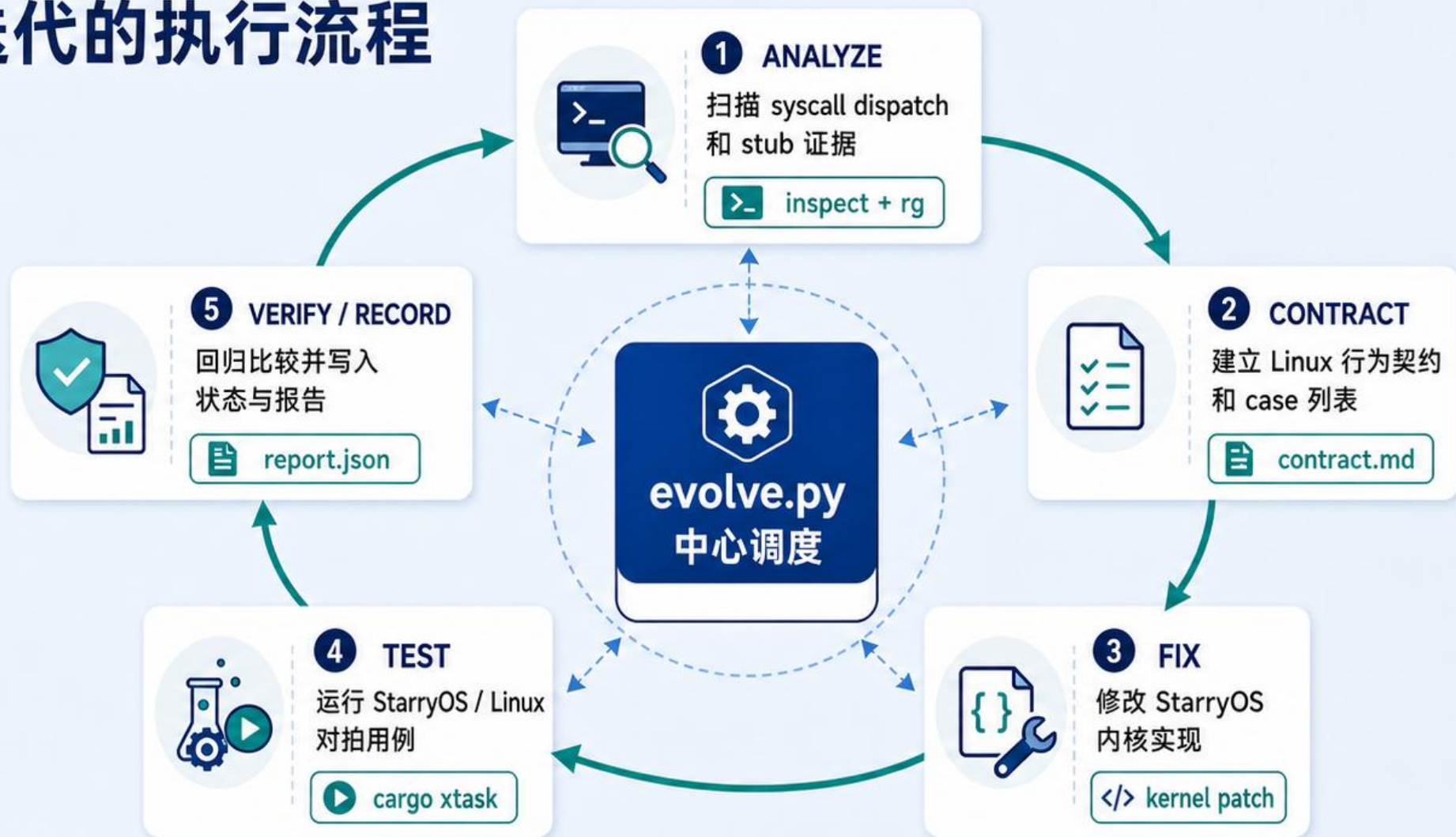
分层原则:

AI 技能层 (橙色)：负责分析代码、生成契约、编写补丁和测试用例。产出物必须经过确定性工具验证。

确定性工具层 (绿色)：解析代码结构、校验契约格式、解析 JSONL、构建报告、对比基线。判定 PASS/FAIL 的逻辑只存在于这一层

产出物 (蓝色)：YAML 契约、JSON 报告、基线文件、状态文件和日志。全部可人工审查。

一轮迭代的执行流程



ANALYZE: 扫描 syscall dispatch 和 stub 证据



CONTRACT: 建立 Linux 行为契约和 case 列表



FIX: 修改 StarryOS 内核实现



TEST / VERIFY / RECORD: 对抽、回归比较并写入状态

Workflow

行为	AI 可做	只有确定性工具可做	防范的风险
选择下一个改进目标	建议	<code>evolve.py select</code> 从审计结果排序	AI 可能跳过真正有问题的 syscall
定义 syscall 契约	起草 YAML	<code>contract_schema.py</code> 校验 schema	AI 可能写出格式不合法或缺少必要字段的契约
编写内核补丁	生成代码	<code>cargo xtask clippy</code> / 编译验证	AI 可能引入编译错误或 clippy 警告
编写测试用例	生成 C 代码	<code>verifier.py</code> 解析 JSONL 并校验 checksum	AI 可能输出无法通过 checksum 校验的伪造结果
判定测试通过	不可	<code>test_runner.py</code> 对比 Linux/StarryOS JSONL	AI 可能误读终端输出或凭印象声称通过
标记 VERIFIED 状态	不可	<code>evolve.py record</code> 从 verifier report 派生	AI 可能绕过验证直接声称完成
写入 journal 成功项	不可	<code>evolve.py record</code> 从 verifier report 派生	同上，防止凭空捏造成功记录

确定性工具与 AI 紧密协作

目标选择：AI 不凭经验手选

利用源码事实驱动的审计与排序，只在 STUB / PARTIAL 上发力，限制 AI 的随意性

syscall_audit.py 事实扫描



IMPLEMENTED

已有完整实现，可直接使用

352

58.7%



PARTIAL

部分实现或 TODO，具备改进空间

167

27.8%



STUB

未实现，仅有 stub，占比最小但价值最大

81

13.5%



扫描范围：syscall/mod.rs **dispatch** 表（按特性条件展开）



evolve.py select 候选目标

1



STUB 优先

完全未实现，收益最高

Evidence: 强

示例：sys_reboot

2



PARTIAL 次之

已有部分实现，补全更高效

Evidence: 中

示例：sys_munmap

3



已实现跳过

不作为 AI 迭代目标

Evidence: 弱

示例：sys_getpid



handler 定位证据



syscall/mod.rs
dispatch 表

提取 syscall id 与入口



handler 文件

定位到具体源码文件
如：fs/open.rs

123

行号

定位精确行号
如：fs/open.rs:42



stub 证据

识别 stub / TODO
/ unimplemented!()



实现状态

归类为 STUB /
PARTIAL / IMPLEMENTED



syscall_audit.py 解析
syscall/mod.rs 的
dispatch 表



定位 **handler** 文件、行号、
stub 证据和实现状态



evolve.py select 基于
STUB / PARTIAL 排序
选择候选目标



候选目标来自**源码事实**，
而不是模型印象



目标来自源码事实
而非模型印象

定义契约

AI 做什么： AI 查阅 Linux man page 和内核源码，为某个 syscall 起草一份 YAML 契约文件，定义若干测试 case 及其 `case_id` 和 `compare` 字段。

工具做什么： `contract_schema.py` 的 `validate_contract` 函数对 YAML 进行严格的结构校验：

- 检查 `schema_version`、`syscall`、`status`、`cases` 四个顶层字段是否存在
- 检查 `status` 是否在允许值集合 `DISCOVERED/ANALYZED/CONTRACTED/IMPLEMENTED/TESTED/VERIFIED/UNSUPPORTED` 中
- 检查 `cases` 列表不为空、每个 case 有唯一的 `case_id`
- 检查每个非 `unsupported` 的 case 在 `compare` 中至少启用了 `ret`、`errno`、`observable` 中的一个
- 检查 `compare` 字典中没有未知的 key（只允许这三个字段）
- 检查 `unsupported` case 必须提供 `reason`
- 检查 `linux_sources` 中每个条目都有 `type` 和 `reference`

如果任何一个校验失败，抛出 `ContractError`，契约被拒绝，无法进入后续的修复和测试阶段。

为什么不能只靠 AI： AI 可能遗漏必要字段（如忘记写 `compare`）、使用非法的 `status` 值、或在同一个契约中定义重复的 `case_id`。工具的校验是确定性的，不会因为疲劳或注意力分散而漏检。

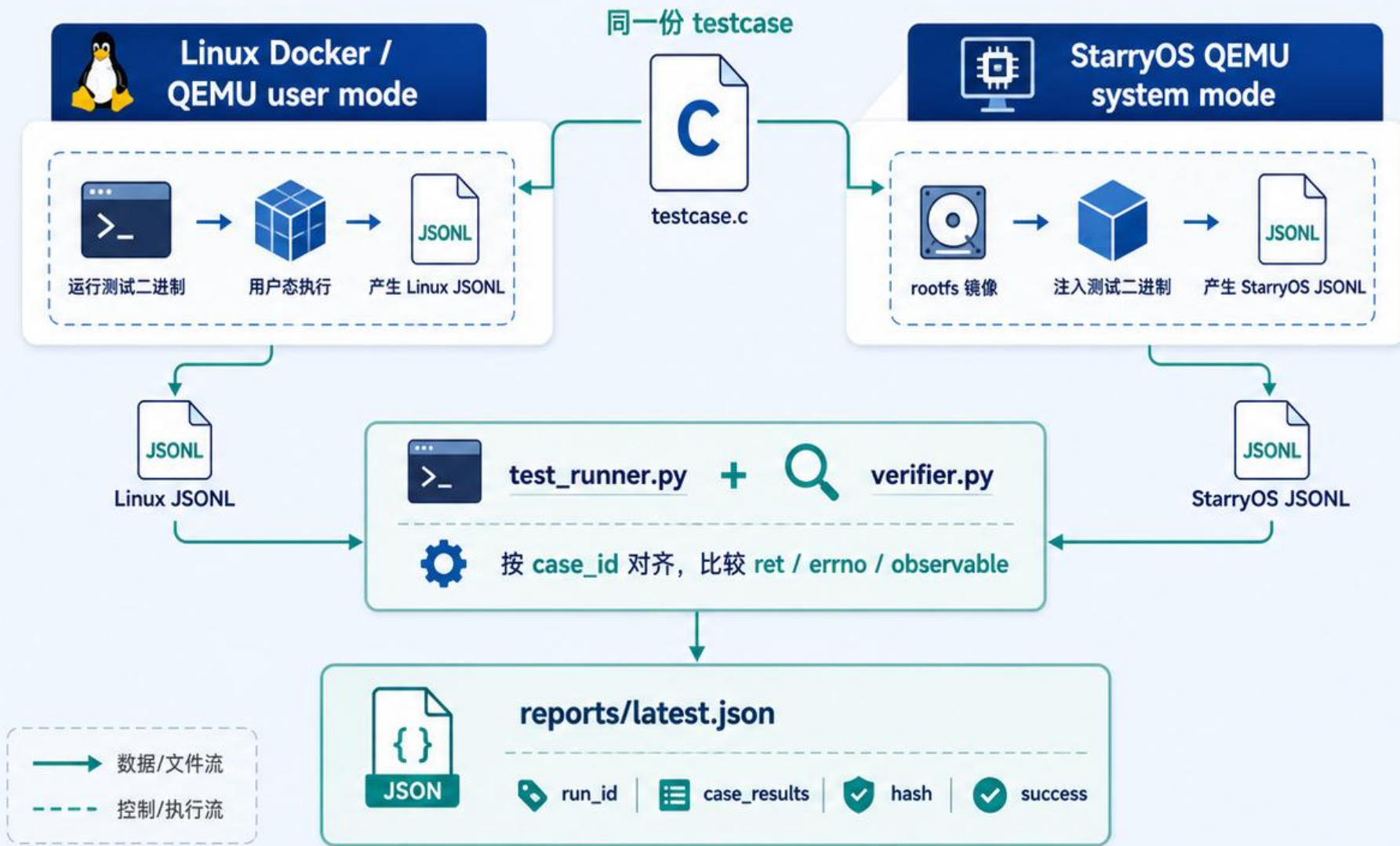
编写内核补丁：AI 生成，编译器验证

AI 做什么： AI 根据 YAML 契约修改 `os/StarryOS/kernel/src/syscall/` 下的 Rust 代码，实现或修复目标 syscall。

工具做什么： `cargo xtask clippy` 和 `cargo xtask starry build` 执行编译。如果 AI 生成的代码有语法错误、类型不匹配、或 clippy 能检测到的惯用法问题，编译会失败，补丁不会进入测试阶段。

Linux Docker vs StarryOS QEMU 对拍验证

同一测试程序，两端运行；Linux 作为 oracle，verifier 决定是否通过



1
Linux 运行结果
作为 **oracle**



2
只接受带 **run_id** 和
checksum 的 JSONL



3
PASS 来自
verifier report,
不来自终端文本

JSONL + checksum: 防止“看起来通过”

1



只接受
type=starry_evolve_case
的 JSONL 记录

2



每条记录包含
run_id、case_id、ret、
errno、observable、
checksum

3



checksum 基于
FNV-1a 64
重新计算，
不匹配就拒绝

4



终端里的
PASS / PASSED
文本会被忽略



结构化 JSONL 记录 (示例)

只信任来自系统的结构化 JSONL 数据

```
{"type":"starry_evolve_case","run_id":1024,"case_id":"fs_ext4_rw","ret":0,"errno":0,"observable":"sha256:ab12...","checksum":"f4c3a9b8e1d2c3b4"}
```

type



starry_evolve_case

run_id



1024

case_id



fs_ext4_rw

ret



0

errno



0

observable



sha256:ab12...

checksum



f4c3a9b8e1d2c3b4

受信任的结构化数据



checksum 校验器



FNV-1a 64
重新计算

计算结果与记录中的
checksum 是否一致?



匹配
接受



不匹配
拒绝



终端文本被忽略
这些文本不作为证据

TEST fs_ext4_rw ... PASS | PASSED | success



判定测试通过：最关键的信任边界

AI 做什么：AI 编写 C 测试程序（如 `testcases/syncfs_pair.c`），该程序在运行时输出结构化 JSONL 记录。

工具做什么：判定过程由 `verifier.py` 的 `parse_jsonl_cases` 函数独立完成，具体执行以下步骤：

第一步：过滤非 JSON 行。原始输出（特别是 StarryOS 的串口输出）会包含大量内核日志、启动信息等噪音。`parse_jsonl_cases` 逐行尝试 `json.loads`，解析失败的行直接跳过。特别地，如果一行包含 `PASS` 或 `PASSED` 文本但不是有效 JSON，它会被记录为 `unstructured success text ignored`——也就是说，**框架显式拒绝终端中的 PASSED 文本**。

第二步：校验 JSONL 记录类型。只接受 `type` 字段为 `"starry_evolve_case"` 的记录。其他 JSON 对象（如内核日志中的 JSON 格式输出）被忽略。

第三步：校验 run_id。每次验证会话生成一个随机 16 字符十六进制 `run_id`，通过环境变量 `STARRY_EVOLVE_RUN_ID` 传入测试程序。`parse_jsonl_cases` 拒绝所有 `run_id` 不匹配的记录。这防止了不同次运行的输出被混在一起，也防止了伪造的旧输出被注入。

第四步：校验 case_id。只接受在契约 YAML `cases` 列表中定义过的 `case_id`。未知的 `case_id` 被拒绝。

第五步：校验 checksum。这是最关键的防篡改步骤。对于每条记录，`verifier.py` 用 `run_id`、`case_id`、`ret`、`errno` 和 `observable` 的规范 JSON 字符串拼接后计算 FNV-1a 64 哈希，然后与记录中的 `checksum` 字段对比。如果两者不一致，该记录被拒绝（标记为 `checksum mismatch`）。

FNV-1a 64 的计算过程（对应 `testcases/syncfs_pair.c` 中的 C 实现和 `verifier.py` 中的 Python 实现）：

```
输入 = run_id + "\n" + case_id + "\n" + str(ret) + "\n" + str(errno) + "\n" + observable_json
哈希 = FNV-1a-64(输入)
```

两端使用完全相同的算法，确保：

判定测试通过：最关键的信任边界

两端使用完全相同的算法，确保：

- 如果 AI 修改了测试程序使其在 StarryOS 端伪造一个 `ret=-1, errno=9` 的结果，但实际 syscall 返回了不同的值，checksum 会对不上（因为 C 程序用实际的 syscall 返回值计算 checksum）。
- 如果有人直接在串口输出中注入一条伪造的 JSONL 行，但不知道当前的 `run_id`，该行会被第三步拒绝。
- 即使知道 `run_id` 并伪造了一条 checksum 正确的记录，仍然需要 Linux 端和 StarryOS 端的对应 case 结果完全一致（见下一步）。

第六步：拒绝重复 case。 同一个 `case_id` 出现多次时，只接受第一次，后续的被拒绝。这防止了通过重复输出来覆盖之前的结果。

第七步：pair 对比。 经过上述过滤后，`verifier.py` 的 `evaluate_pairwise` 函数按 `case_id` 将 Linux 端和 StarryOS 端的结果配对，然后按照契约 `compare` 字段中启用的字段逐一对比。**Linux 端是 oracle（参考基准）**——如果 StarryOS 的 `ret`、`errno` 或 `observable` 与 Linux 不一致，该 case 被标记为 `FAIL`。

如果所有有效 JSONL 都被拒绝怎么办？ 如果原始输出中存在 `PASS / PASSED` 文本，但没有任何一条 JSONL 记录通过了上述校验（`accepted` 为空），那些 `PASS / PASSED` 文本会被追加到 `rejected` 列表中，最终判定为失败。换句话说，**没有有效 JSONL 支撑的 PASSED 声明等同于失败。**

verifier report: 每次验证都可追溯

1 身份: syscall / target / run_id



syscall	openat
target	starryos
run_id	2025-05-29T10:25:18Z

2 命令: contract / source / binary



contract	contracts/openat.toml
source	starry-evolve@a1b2c3d
binary	build/starryos-evolve

3 case 结果: case_results / 退出码 / rejected lines / case 匹配



退出码	0 (success)
rejected lines	2 / 231
case_results	12 passed / 12
case 匹配	12 / 12 (100%)



reports/latest.json

类型: report 版本: 1.0

```
{  "identity": {    "syscall": "openat",    "target": "starryos",    "run_id": "2025-05-29T10:25:18Z"  },  "command": {    "contract": "contracts/openat.toml",    "source": "starry-evolve@a1b2c3d",    "binary": "build/starryos-evolve"  },  "case_results": {    "exit_code": 0,    "rejected_lines": 2,    "case_matched": 12,    "case_total": 12,    "case_matal_ratio": 1.00  },  "hash": {    "git_diff_hash": "d4f6c9b2e7a1...",    "raw_log_hash": "7a8e3d1c9b0f...",    "fingerprint": "FPR-3A17-9C2B-5E6D"  },  "record_check": {    "evolve_record": true,    "schema_valid": true,    "current_diff_hash_match": true  }}
```

4 hash: git diff hash / raw log hash / fingerprint



git diff hash	d4f6c9b2e7a1...
raw log hash	7a8e3d1c9b0f...
fingerprint	FPR-3A17-9C2B-5E6D

5 record 校验: evolve.py record / schema 校验 / 当前 diff hash



evolve.py record	true
schema 校验	通过
当前 diff hash	匹配



原始日志
完整保存



git diff
精确定位



结果可审计
可复现



审计要点

1



reports/latest.json
记录 syscall、
target、run_id、
case_results

2



记录 contract、
source、binary、
git diff、raw log hash

3



success 由退出码、
rejected lines 和
case 匹配共同决定

4



evolve.py record 会
校验 schema 和
当前 git diff hash



可追溯 · 可复现 · 可审计

每一次验证都生成完整的 report, 不仅告诉你结果, 还告诉你“是谁、做了什么、用了什么、如何决定、如何复现”。

标记 VERIFIED 状态和写入 journal: 只有 report 可以触发

AI 做什么：不能做任何事。VERIFIED 状态不能由 AI 直接写入。

工具做什么：`evolve.py record --report <path>` 是唯一能更新状态的路径。它执行三重校验：

1. **字段完整性校验：**检查 report JSON 是否包含 `schema_version`、`run_id`、`syscall`、`target`、`success`、`git_diff_hash`、`case_results` 这七个必要字段。缺少任何一个，直接报错退出。
2. **Schema 版本校验：**检查 `schema_version` 是否等于当前工具支持的版本（`REPORT_SCHEMA_VERSION = 1`）。如果不匹配，报错退出——防止使用旧版本工具生成的 report。
3. **git diff hash 校验：**这是防止「过期 report」的关键。`verifier.py` 在生成 report 时会计算当前工作区 `git diff --binary HEAD` 的 SHA-256 哈希并写入 `git_diff_hash` 字段。`evolve.py record` 在记录时会重新计算当前工作区的 git diff hash，如果两者不匹配——意味着 report 对应的代码状态和当前代码状态不一致——则拒绝记录。

只有通过这三重校验后，`record` 才会：

- 将 `syscall_status.yaml` 中该 syscall 的 `status` 设为 `VERIFIED`（如果 report 中 `success=true`）或 `TESTED`（如果 `success=false`）
- 在 `journal.md` 中追加一条包含 `run_id`、`target`、`diff_hash`、两端 raw log hash 的记录
- 记录每个 case 的通过/失败状态到 `case_status` 字典

为什么不允许 AI 手写 VERIFIED：如果允许 AI 直接在 `syscall_status.yaml` 或 `journal.md` 中写入「VERIFIED」，那么整个确定性验证链就失去了意义——AI 可以在不运行任何测试的情况下声称某个 syscall 已经验证通过。强制要求从 verifier report 派生状态，保证了每一个 VERIFIED 标记都可以追溯到一次真实的对拍运行。

契约格式

契约以 YAML 文件存储在 `scripts/starryevolve/contracts/<syscall>.yaml`

示例 (`contracts/syncfs.yaml`):

```
schema_version: 1
syscall: syncfs
status: CONTRACTED
linux_sources:
  - type: man-pages
    reference: "syncfs(2)"
    note: "syncfs(fd) synchronizes the filesystem containing fd."
  - type: linux-kernel
    reference: "fs/sync.c: SYSCALL_DEFINE1(syncfs, int, fd)"
    note: "Linux validates fd and returns EBADF for invalid descriptors."
cases:
  - case_id: syncfs_invalid_fd_ebadf
    description: "syncfs(-1) fails with EBADF"
    compare:
      ret: true
      errno: true
      observable: true
```

关键字段说明:

- `schema_version`: 契约格式版本, `contract_schema.py` 据此校验。
- `cases[].case_id`: 测试 case 的唯一标识, C 测试程序和 verifier 通过此 ID 对齐。
- `cases[].compare`: 指定哪些字段参与 Linux/StarryOS 对比 (`ret`、`errno`、`observable`)。
- `cases[].unsupported`: 可选, 标记 StarryOS 预期不支持的 case, 对拍时跳过。
- `linux_sources`: 信息性字段, 记录 Linux 行为的参考来源。Linux 的实际运行时行为由 Docker 对拍确定, 不在 YAML 中手写 oracle。

校验契约:

```
python3 scripts/starry-evolve/evolve.py contract --syscall syncfs
```

Json 格式

```
{ "type": "starry_evolve_case",  
  "run_id": "a1b2c3d4e5f6a1b2",  
  "case_id": "syncfs_invalid_fd_ebadf",  
  "ret": -1,  
  "errno": 9,  
  "observable": { "error": "EBADF" },  
  "checksum": "a1b2c3d4e5f6a1b2" }
```

字段说明:

- `type`: 必须为 `starry_evolve_case`, 其他 JSON 行被忽略。
- `run_id`: 与当前验证会话的环境变量匹配, 防止混入无关输出。
- `case_id`: 对应契约中的 case 标识。
- `ret`: syscall 返回值。
- `errno`: errno 值。
- `observable`: 自由格式的 JSON, 由契约 `compare.observable` 决定是否参与对比。
- `checksum`: FNV-1a 64 哈希, 输入为 `run_id\ncase_id\nret\nerrno\nobservable_json`。

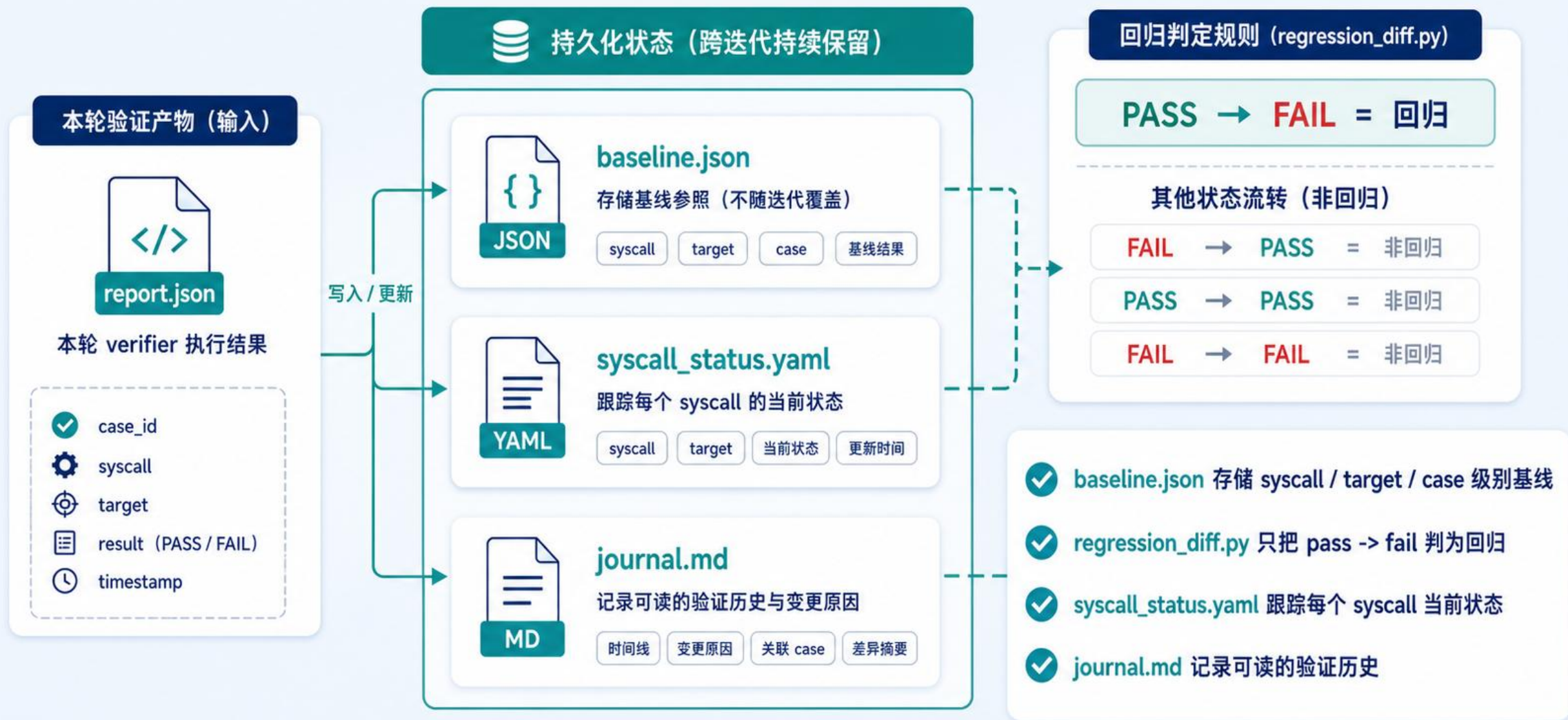
6.3 拒绝规则

`verifier.py` 的 `parse_jsonl_cases` 会拒绝以下记录:

- `type` 不是 `starry_evolve_case`
- `run_id` 不匹配当前会话
- `case_id` 不在契约定义中
- `ret` 或 `errno` 无法解析为整数
- `checksum` 与重算结果不一致
- 同一 `case_id` 出现多次

如果原始输出中包含 `PASS / PASSED` 文本但没有任何有效的 JSONL 记录, 这些文本会被报告为 `unstructured success text ignored`, 最终判定为失败。

状态流转：baseline、status、journal



AI-coding 集成: prompt、skills、agents、hooks

给 AI-coding agent 运行的 StarryOS 持续改进框架



Loop&subagent

8.2 迭代循环

`starry-iterate` 编排完整循环：

```
ANALYZE → CONTRACT → FIX → TEST → VERIFY → RECORD
```

可与 `/loop` 配合实现多轮自主改进。每轮只修改一个 syscall，失败不超过 2 次重试。

8.3 子 Agent

迭代流程中可派生两个子 Agent：

- `regression-watcher.yaml`：检测内核变更引入的回归，报告 BUILD/VERIFIER_REPORT/REGRESSIONS/IMPROVEMENTS。
- `syscall-reviewer.yaml`：对照契约 YAML 审查 syscall 实现，检查参数处理、错误码、返回值、用户空间内存访问和边界情况。

两个子 Agent 的输出必须引用契约 `case_id`，不允许无依据的 PASS 判定。

Skills&Hook

9.2 Hook

事件	Hook	职责
SessionStart	hooks/session_context.py	加载会话上下文
Stop	hooks/pre_commit_check.py	在 Agent 停止前检查是否有未验证的内核变更，防止提交未通过 verifier 的修改

pre_commit_check.py 作为安全网：如果 Agent 尝试在内核变更未经 verifier 验证的情况下结束会话，hook 会发出警告。

技能	职责	触发条件
starry-analyze	审计 syscall 覆盖率，分类实现状态	"analyze starry"、"syscall audit"
starry-contract	提取 Linux 行为契约，生成 YAML	"check contract"、"对比 Linux 行为"
starry-fix	根据契约实现或修复 syscall	"fix sys_X"、"implement syscall"
starry-test	生成和运行 JSONL 测试用例	"test syscall"、"write test"
starry-verify	跨架构构建和回归验证	"verify starry"、"run regression"
starry-iterate	完整自主迭代循环	"iterate on starry"、"持续改进"



Demo展示



章节

chapter

BiglabB T2

—
成果展示

PR	状态	问题	关键点
#224	已合并	signal check 线程串扰	<code>block_next_signal</code> 从全局状态改为线程私有
#225	关闭未合并	<code>/proc/status</code> affinity 固定 CPU0	初版修复, 后续由 #267 收敛
#267	已合并	<code>/proc/status</code> affinity 错误	动态输出 <code>Cpus_allowed</code> / <code>Cpus_allowed_list</code>
#276	已合并	<code>sched_*affinity(pid)</code> 忽略 pid	支持查询/设置目标任务 affinity
#475	关闭未合并	BusyBox iostat 缺 <code>/proc</code> 统计	补 <code>/proc/stat</code> 、 <code>diskstats</code> 、 <code>uptime</code>
#659	已合并	<code>sync()</code> 仅 stub	接入 VFS root sync
#660	已合并	<code>syncfs(fd)</code> 仅 stub	定位 fd 所属文件系统并 sync
#689	已合并	PicoClaw Phase 1/2 支持	离线 CLI、在线 agent smoke
#775	已合并	PicoClaw gateway 支持	gateway/interactive QEMU 配置
#776	已合并	readline/TUI 卡在 <code>ESC[6n</code>	tty 注入光标位置响应
#780	已合并	Go runtime <code>PR_SET_VMA</code> 报错	<code>PR_SET_VMA_ANON_NAME</code> silent no-op
#781	已合并	<code>waitid()</code> 未实现	支持 <code>P_ALL/P_PID</code> 、 <code>WEXITED/WNOHANG/WNOWAIT</code>
#875	开放中	缺 GNU diffutils app 覆盖	新增 <code>apps/starry/diffutils</code> 多架构测例
#894	已合并	<code>inotifywait</code> 不可运行	实现 <code>inotify fd/watch/event/read/poll</code>
#994	开放中	K230 KPU QEMU 第一版适配	静态 K230 平台、 <code>/dev/kpu</code> 、KPU smoke
#1036	Draft	K230 KPU 动态平台适配	FDT/rdrive 路线、SDHCI、NNCase runtime 链路

- 目前已合并PR 11个:
- 修复StarryOS BUG 3个
- 支持syscall 3个
- 支撑Linux 小粒度应用: `diff`,`inotifywait`
- 完成StarryOS对Picoclraw的支持
- 完成StarryOS对 QEMU K230 KPU 的设备适配

1 问题背景

1.1 inotifywait 依赖哪些 Linux 内核能力

Alpine Linux 的 `inotify-tools` 包提供 `inotifywait` 命令行工具，其工作流程为：

1. 调用 `inotify_init1(flags)` 获取一个 inotify 文件描述符；
2. 调用 `inotify_add_watch(fd, path, mask)` 注册一个或多个监控路径；
3. 对该 fd 执行 `poll`（等待可读）、`ioctl(FIONREAD)`（探测可读字节数）、`read`（读取 `inotify_event` 结构体序列）；
4. 使用完毕后调用 `inotify_rm_watch` 移除监控，最终 `close(fd)`。

其中 `inotify_event` 的结构（`<sys/inotify.h>`）为：

```
struct inotify_event {  
    int      wd;          /* watch descriptor */  
    uint32_t mask;        /* 事件掩码 */  
    uint32_t cookie;      /* rename 关联 cookie (本 PR 始终为 0) */  
    uint32_t len;         /* name 字段的长度 (含 padding) */  
    char     name[];      /* 可选：被监控目录下的子文件名 */  
};
```

1.2 StarryOS 原有的处理方式

在此 PR 之前，StarryOS 对 `inotify_init1` 仅返回一个 dummy fd，不实现任何语义。`inotify_add_watch` 和 `inotify_rm_watch` 则完全缺失。

这意味着：

- `inotifywait` 调用 `inotify_init1` 得到一个 fd，但后续对该 fd 的 `poll` 永远不返回可读、`read` 行为未定义；
- 即使 `inotify_add_watch` 能注册路径，内核侧也缺乏在文件系统操作发生时向 `inotify` fd 推送事件的机制。

`inotifywait` 是一个真实 Linux 用户态工具，它的基本流程是：先调用 `inotify_init1` 得到一个 fd，然后调用 `inotify_add_watch` 监听某个路径，之后阻塞在 `poll` 或 `read` 上，等待内核在文件发生修改、创建、关闭写入、删除时往这个 fd 里投递 `inotify_event`。

StarryOS 原来的问题是，`inotify_init1` 更接近 dummy fd，用户态能拿到 fd，但这个 fd 背后没有 watch 表，没有事件队列，也不能在文件系统操作发生时被唤醒。所以工具会超时，表现为 `inotifywait` 观察不到事件。

所以对StarryOS提出了更高的要求

具体实现(数据结构)

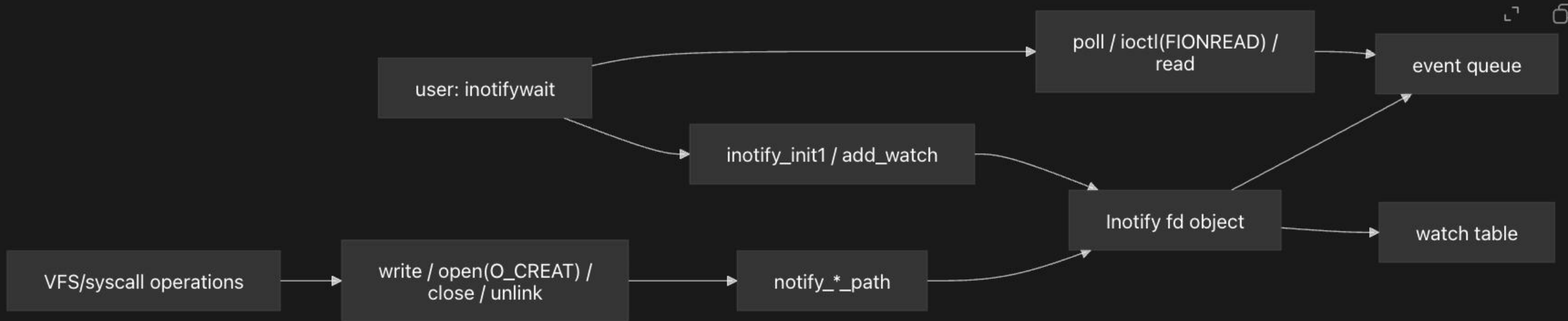
```
// Watch 条目：一个绝对路径 + 事件掩码
struct Watch {
    path: String,
    mask: u32,
}

// inotify 实例的内部状态
struct InotifyState {
    next_wd: i32,                // 下一个可分配的 watch descriptor
    watches: BTreeMap<i32, Watch>, // wd → Watch 映射
    queue: VecDeque<Vec<u8>>,    // 已序列化的事件队列
}

// 对外暴露的 inotify 文件对象
pub struct Inotify {
    non_blocking: AtomicBool,
    state: Mutex<InotifyState>,
    poll_rx: PollSet,           // 用于唤醒 poll 等待者
}
```

内核需要在文件变化时主动通知用户态。这个 PR 的核心就是给 StarryOS 补上“文件系统事件订阅与通知”这条机制。

层次	行为
用户态需求	应用希望监听“某个文件/目录发生变化”
内核抽象	内核提供一个特殊 fd，用户把 watch 注册到这个 fd 上
事件来源	文件系统操作发生时，内核识别出 modify/create/delete/close+write
通知机制	内核把事件放到 fd 上，并唤醒等待的应用



还有新的挑战

第一，内核要能记录用户态订阅了什么。也就是用户调用 `inotify_add_watch` 后，内核保存“哪个 fd 在监听哪个路径、关心哪些事件”。

第二，文件系统操作发生时，内核要能识别事件。比如写文件对应 `MODIFY`，新建文件对应 `CREATE`，可写文件关闭对应 `CLOSE_WRITE`，删除文件对应 `DELETE`。

第三，事件发生后，内核要把它送回用户态。也就是把事件挂到对应的 `inotify fd` 上，并唤醒正在等待的 `inotifywait`，让它读到事件并继续执行

用户态:

```
inotify_add_watch(fd, "/tmp/dir", IN_CREATE)
```

|
v

内核记录:

```
fd -> watch table:
```

```
wd=1, path="/tmp/dir", mask=IN_CREATE
```

之后文件系统发生操作:

```
open("/tmp/dir/a.txt", O_CREAT)
```

|
v

内核识别:

这是 CREATE 事件, 事件路径是 /tmp/dir/a.txt

|
v

内核匹配:

watch path="/tmp/dir" 是事件路径的父目录

mask 包含 IN_CREATE

|
v

内核通知:

把事件 {wd=1, mask=IN_CREATE, name="a.txt"} 放入 fd 队列

唤醒等待 fd 的 inotifywait

|
v

用户态:

```
read(fd) 得到事件, 打印 CREATE
```

这个机制分成三部分: watch 表保存订阅关系, VFS 操作路径产生事件, inotify fd 事件队列负责把事件返回给用户态。

1.内核会记录用户订阅了什么: 内核根据 fd 找到对应的 Inotify 对象, 把用户传入的 path 解析成内核里的绝对路径, 在这个 Inotify 对象的 watch 表里保存

2.文件系统操作发生时, 内核识别事件。

3.事件发生后, 内核会遍历当前所有 inotify fd, 检查哪些 watch 和这个路径、事件类型匹配。匹配成功后, 就把事件放进对应 fd 的事件队列, 并唤醒等待者。于是用户态的 inotifywait 从睡眠中返回, 再通过 read 读到标准的 inotify_event。



Chapter

Chapter

BigLabB T4-1

—

Picoclave

PicoClaw的简要介绍



PicoClaw 是一款基于 NanoBot 设计的超轻量级个人人工智能助手。它完全由零开始，借助AI用 Go 语言重新构建而成。

得益于Go语言的高效和精简设计，PicoClaw的二进制文件体积小巧，运行时内存和CPU消耗极低，使其能够轻松运行在树莓派、嵌入式开发板乃至老旧电脑上。

四阶段工作

Phase 1: 离线 Smoke

目标：验证静态 Linux x86_64 PicoClaw 二进制能在 StarryOS x86_64 QEMU 中启动、写入配置并执行本地 CLI 命令。

Phase 2: 在线 Agent

目标：注入 API key、代理和 CA 后，验证 `picoclaw agent -m ...` 能完成一次真实模型请求。

Phase 3: Gateway 服务

目标：验证 `picoclaw gateway` 能在 StarryOS guest 中启动长时间运行的 HTTP 服务，并响应本地检查。

Phase 4: TTY/readline 交互修复

目标：让裸 `picoclaw agent` 能直接读取键盘输入，完成多轮对话并用 `exit` 退出。

成果展示

支持 cli-agent

```
root@starry:/root/.pico claw/workspace # pico claw gateway
[ 27.893211 0:35 starry_kernel::syscall::task::ctl:152] sys_prctl: unsupported option 1398164801
```

PICOCLAW

Agent Status:

- Tools: 16 loaded
- Skills: 0/0 available

✓ Cron service started

✓ Heartbeat service started

07:16:09 WRN channels ../../home/runner/work/pico claw/pico claw/pkg/channels/base.go:131 > SECURITY: Channel allows EVERYONE (allow_from is empty) channel=weixin hint="Set allow_f"

✓ Channels enabled: [weixin]

✓ Health endpoints available at http://127.0.0.1:18790/health, /ready and /reload (POST)

✓ Gateway started on 127.0.0.1:18790

Press Ctrl+C to stop

```
root@starry:/root/.pico claw/workspace # pico claw agent
[ 5.490879 0:24 starry_kernel::syscall::task::ctl:152] sys_prctl: unsupported option 1398164801
```

PICOCLAW

👉 Interactive mode (Ctrl+C to exit)

👉 You: nihao

👉 你好呀! 😊 很高兴见到你! 有什么需要帮忙的吗? 随时告诉我~

👉 You: nishi shi

👉 你好! 😊 你是在说"你是谁"吗?

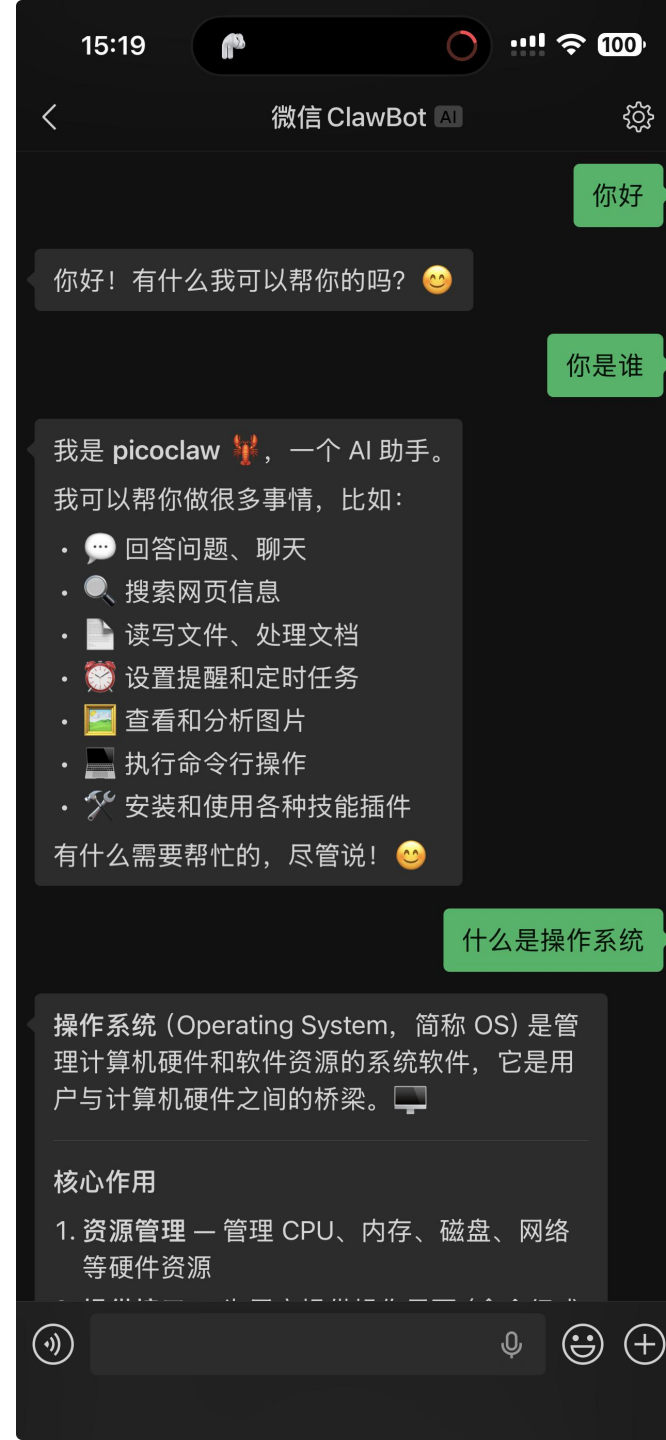
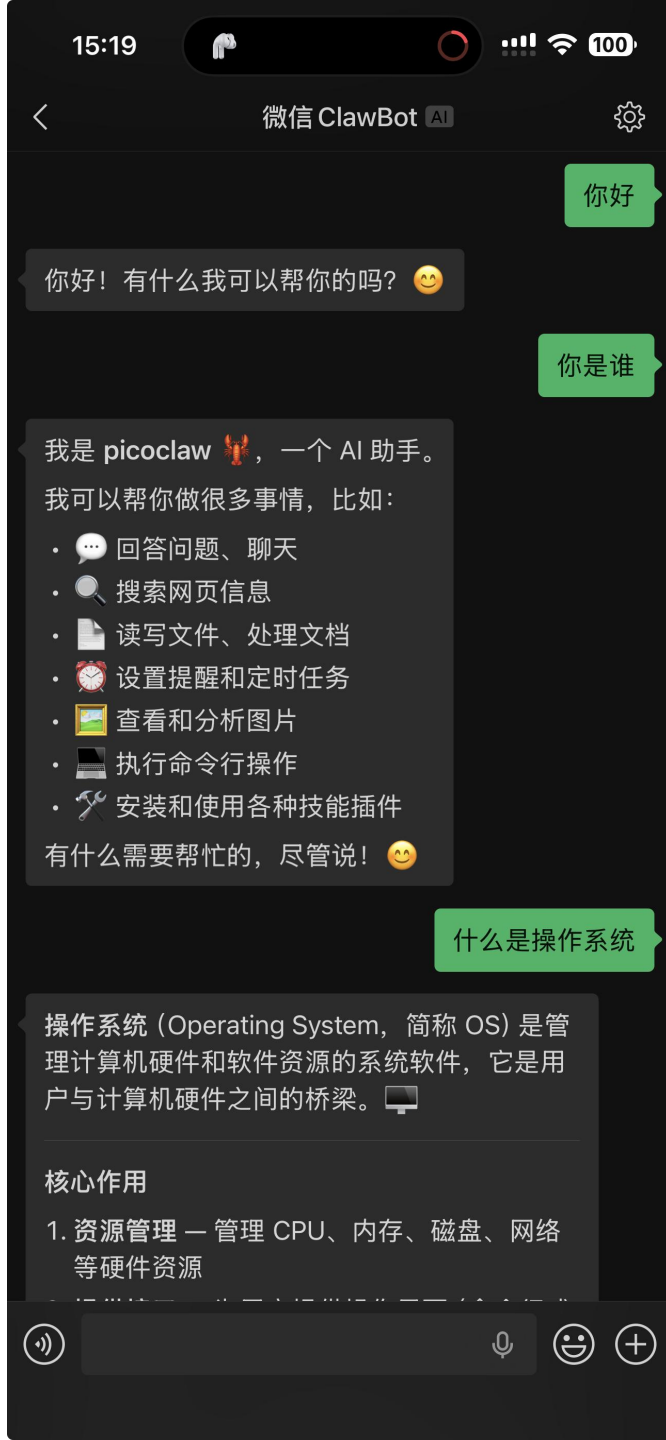
我是 ****pico claw**** 👉, 一个 AI 助手! 我可以帮你做很多事情, 比如:

- 📁 读写文件
- 🔍 搜索网络信息
- 📄 执行命令
- 📅 设置提醒
- 🗂 管理工作区
- 🌐 获取网页内容

有什么我可以帮到你的吗? 随时说! 😊

👉 You: exit

支持微信连接 claw



PR #776 — Phase 4 TTY/readline 修复



- 链接: <https://github.com/rcore-os/tgoskits/pull/776>
- 标题: `fix(starry-kernel): handle tty cursor position report`
- 分支: `codex/phase4-starry-tty-readline` → `dev`
- 状态: 已合并 (2026-05-21)
- 主要 commit:
 - i. `fix(starry-kernel): handle tty cursor position report`

解决了什么问题

Phase 3 之后, `picoclaw agent -m "消息"` 可以正常完成单轮请求, 但直接运行 `picoclaw agent` (不带 `-m`) 进入交互模式后, 键盘输入完全无响应——程序停在 `Interactive mode (Ctrl+C to exit)`, 无论怎么按键都没有反应。这意味着 PicoClaw 在 StarryOS 中只能作为"一次性问答工具"使用, 无法进行多轮交互对话。

一个重要的线索是: `echo "消息" | picoclaw agent` 和 `cat | picoclaw agent` 都可以正常工作。这说明 PicoClaw 的 agent 主循环和网络请求路径没问题, 问题出在 **tty/readline 层**——只有当 stdin 是 tty 时才触发的初始化路径有 bug。

具体是怎么解决的

诊断: readline 初始化卡在等待光标位置响应

PicoClaw 使用 readline 库处理交互式输入。readline 初始化时进入 raw terminal 模式 (关闭 ICANON/ECHO), 然后发送 ANSI Device Status Report 查询 `ESC[6n` ("终端, 告诉我光标在哪"), 等待终端回送形如 `ESC[row;colR` 的响应。收到响应后 readline 才能确定光标位置、初始化行编辑缓冲区。

在 StarryOS 的 QEMU 串口环境中, `ESC[6n` 被写出后:

- 字节通过 tty `write_at()` → `write_output_bytes()` → 串口设备 → QEMU nographic 输出
- 但**没有任何回环路径**: QEMU 串口不是真正的终端模拟器, 不会自动回送 cursor position response
- readline 的 `read()` 调用永远等不到响应数据, 整个线程阻塞在 read 上

这解释了为什么管道模式可以工作: `cat | picoclaw agent` 中 stdin 是 pipe 而非 tty, PicoClaw 检测到 `!isatty(stdin)` 后跳过 readline 初始化, 直接从管道读取。



考虑到 StarryOS QEMU 环境没有真实的终端模拟器来响应 ANSI 查询，修复的核心思路是在内核 **tty 驱动层** 拦截光标位置查询并注入虚拟响应。具体做法分两部分：

第一部分：tty write 路径检测 + 注入（tty/mod.rs）

```
const ANSI_CURSOR_POSITION_REQUEST: &[u8] = b"\x1b[6n";
const ANSI_CURSOR_POSITION_RESPONSE: &[u8] = b"\x1b[1;1R";
```

在 `Tty::write_at()` 中，先正常调用 `write_output_bytes()` 将输出字节写入串口，然后检查输出缓冲区是否包含 `ESC[6n`：

```
if contains_bytes(buf, ANSI_CURSOR_POSITION_REQUEST) {
    self.ldisc.lock().inject_input(ANSI_CURSOR_POSITION_RESPONSE);
}
```

`contains_bytes()` 是新增的简单辅助函数，用 `滑动窗口` 在字节序列中搜索子序列。

注入的响应是固定的 `ESC[1;1R`（光标在第 1 行第 1 列）。这对于 PicoClaw 的 readline 来说足够了——它只需要知道一个合法的光标位置来初始化。

第二部分：line discipline 注入输入队列（tty/terminal/ldisc.rs）

tty 的正常输入路径是从串口硬件 → ring buffer → `read()`。但内核注入的字节不是从硬件来的，需要一个旁路通道。新增 `injected_input: VecDeque<u8>` 字段：

- `inject_input(&mut self, bytes: &[u8])`：将字节追加到 `injected_input` 尾部，然后唤醒 `input_ready`（让阻塞在 `poll()` 的线程知道有数据可读）
- `read(&mut self, buf: &mut [u8])`：优先读取 **injected_input**（在 ring buffer 之前），确保内核注入的响应比用户输入先被读到
- `poll_read(&self) -> bool`：injected_input 非空时立即返回 `true`，确保 `poll()` 不会误报不可读
- `drain_input(&mut self)`：同时清空 ring buffer 和 injected_input（处理 `TCSETSF` 等 flush 操作）

新增单元测试 `injected_input_is_readable_immediately` 验证注入 `ESC[1;1R` 后立即可以被 `read()` 读出。



Demo演示





Chapter

chapter

BigLabB T4-2

KPU on StarryOS

从设备到模型： StarryOS 的 K230 KPU 支持



QEMU K230 上接入 **KPU/NPU**



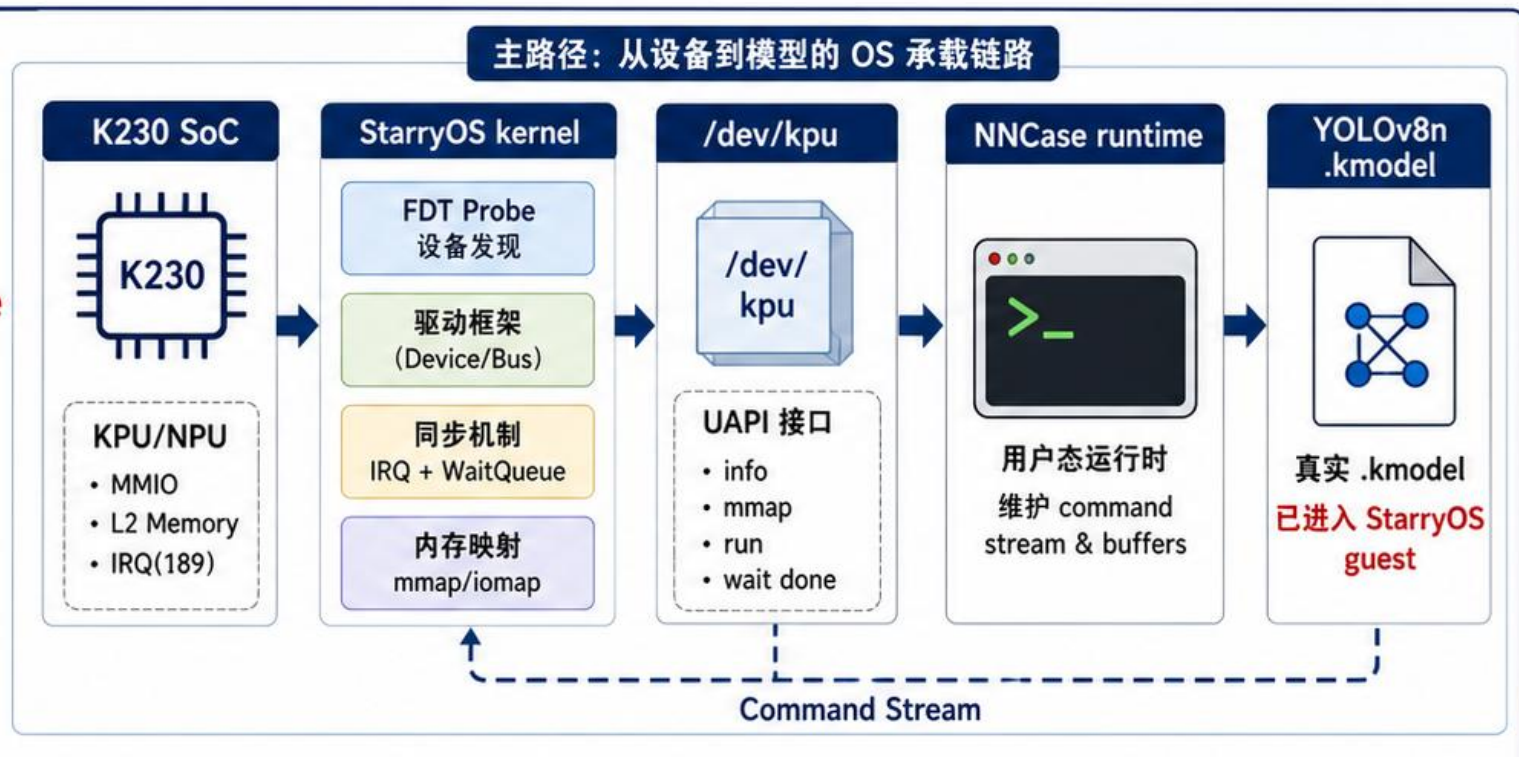
FDT 发现、内核驱动、**/dev/kpu**、**IRQ/done**



NNCase runtime 在 StarryOS
内加载真实 **.kmodel**



边界：YOLO 检测框语义仍在对齐



当前边界：YOLO 检测框语义仍在对齐中（**非** OS 能力问题）

汇报主线：补齐 OS 支撑链路

本次汇报路线

① 设备发现

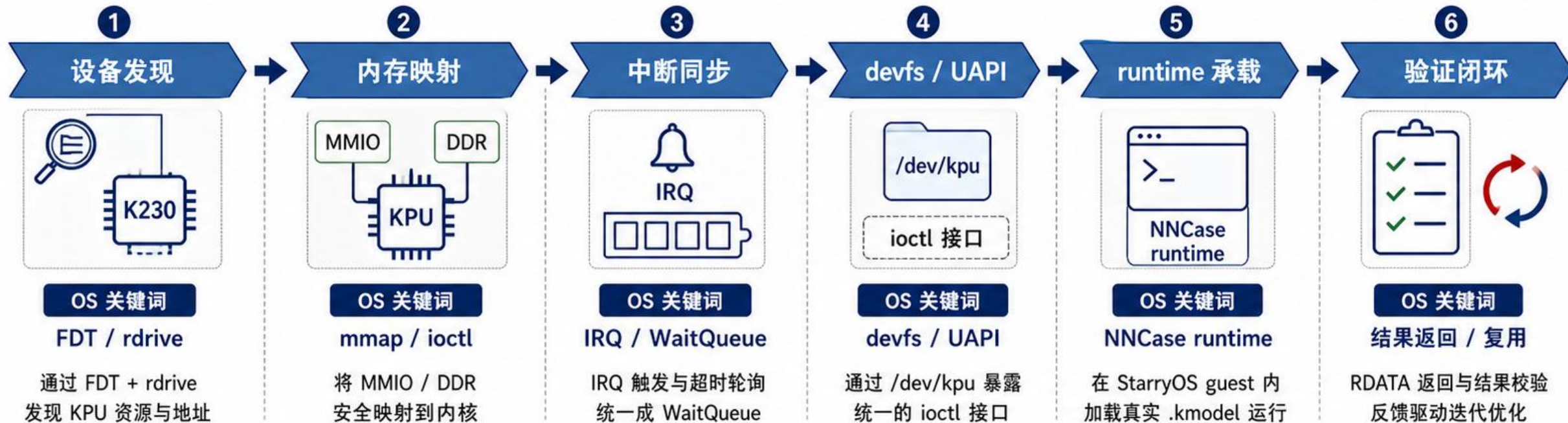
② 内存映射

③ 中断同步

④ devfs / UAPI

⑤ runtime 承载

⑥ 验证闭环



跨阶段
支撑要素



FDT / rdrive
设备描述与资源拓补



mmap / ioctl
资源映射与内核控制通道



IRQ / WaitQueue
事件同步与超时轮询



NNCase runtime
真实 .kmodel 执行与验证

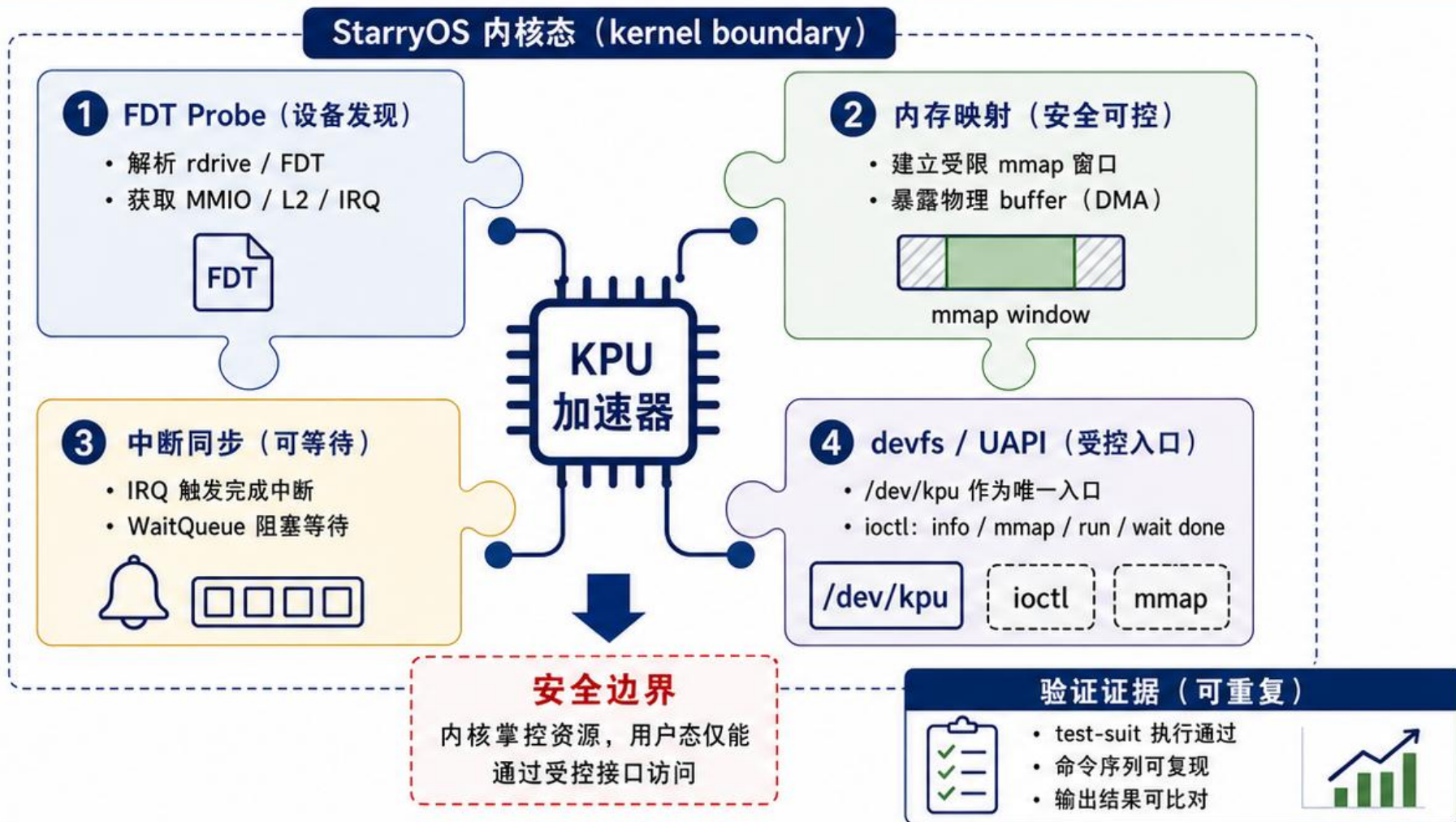


核心结论：沿着 OS 支撑链路逐步补齐，KPU 能力在 StarryOS 内可见、可用、可验证。

支持一个 NPU，OS 层真正要解决什么

本页要点

- 1 硬件资源：**MMIO、L2、IRQ、物理 buffer
- 2 受控入口：**避免用户态任意访问物理内存
- 3 稳定 ABI：**info、mmap、run、wait done
- 4 可重复验证：**test-suite 变成证据

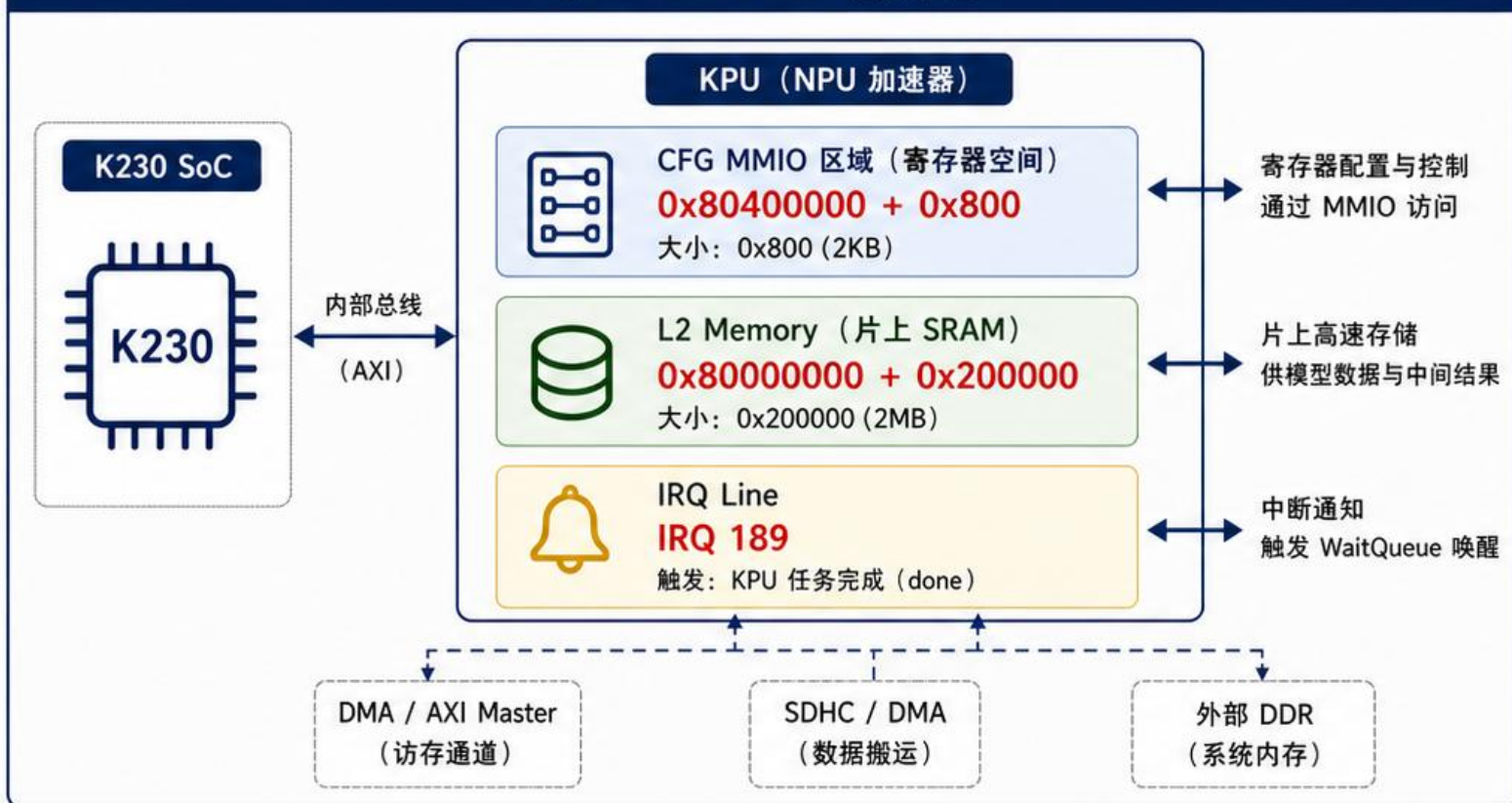


核心结论：适配 KPU 的本质，是内核提供资源发现、安全映射、可等待同步与稳定 ABI。

硬件事实：QEMU K230 暴露的 KPU 边界

- KPU CFG MMIO: $0x80400000 + 0x800$
- KPU L2 memory: $0x80000000 + 0x200000$
- KPU IRQ: 189
- command start/end/high 与 control/status registers

QEMU K230 SoC 设备视图



KPU 寄存器语义 (部分)

寄存器	含义
KPU_CFG_START_ADDR	命令流起始地址
KPU_CFG_END_ADDR	命令流结束地址
KPU_CFG_HIGH_ADDR	命令流高地址
KPU_CFG_CONTROL	启动 / 配置控制
KPU_CFG_STATUS	状态与错误信息
KPU_CFG_IRQ_ENABLE	中断使能控制
KPU_CFG_IRQ_RAW	中断原始状态
KPU_CFG_IRQ_MASK	中断屏蔽掩码
KPU_CFG_RESET	模块复位控制

寄存器访问方式

- 通过 CFG MMIO 区域访问: $0x80400000 \sim 0x80400800$
- 寄存器按 32-bit 对齐, Little-Endian



核心结论：OS 首先面对硬件事实：**地址、大小、IRQ 与寄存器语义**；后续驱动、mmap 与 ioctl 均围绕这些事实展开。

上游约束：迁移到 RISC-V 动态平台

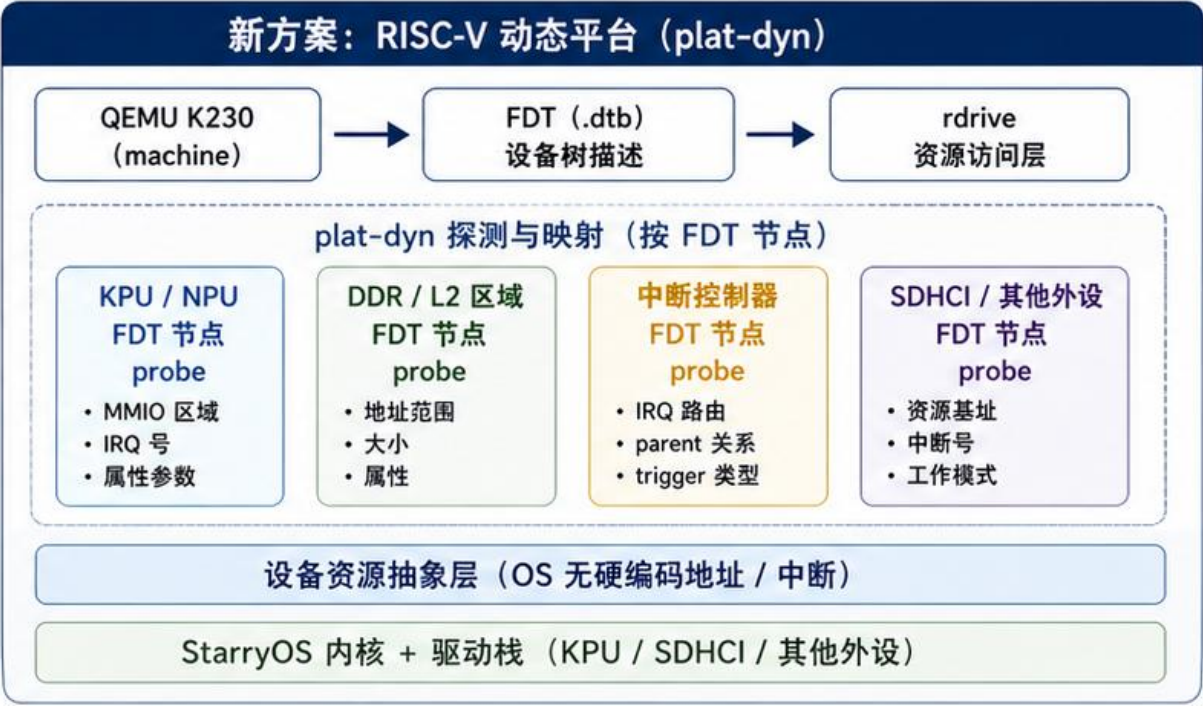
PR 评论要求：将适配从旧的静态 K230 platform 迁移到 riscv64 动态平台，符合 upstream/dev 方向。

- ✓ PR 评论要求迁移到 riscv64 动态平台
- ✓ 使用 plat-dyn + FDT + rdrive 获取设备资源
- ✓ 减少旧静态 K230 platform 依赖
- ✓ 进入 upstream/dev 的长期维护路线



按 upstream/dev 要求迁移

- 以 FDT 为权威描述
- rdrive 提供资源访问
- 平台无硬编码依赖
- 更易跟随上游演进



收益：对齐 upstream/dev 动态平台路线，减少平台耦合，提升可维护性与可移植性



对齐方向
跟随上游
演进更顺畅



降低耦合
减少静态平台
硬编码依赖



提升维护性
新 SoC / QEMU
更易适配



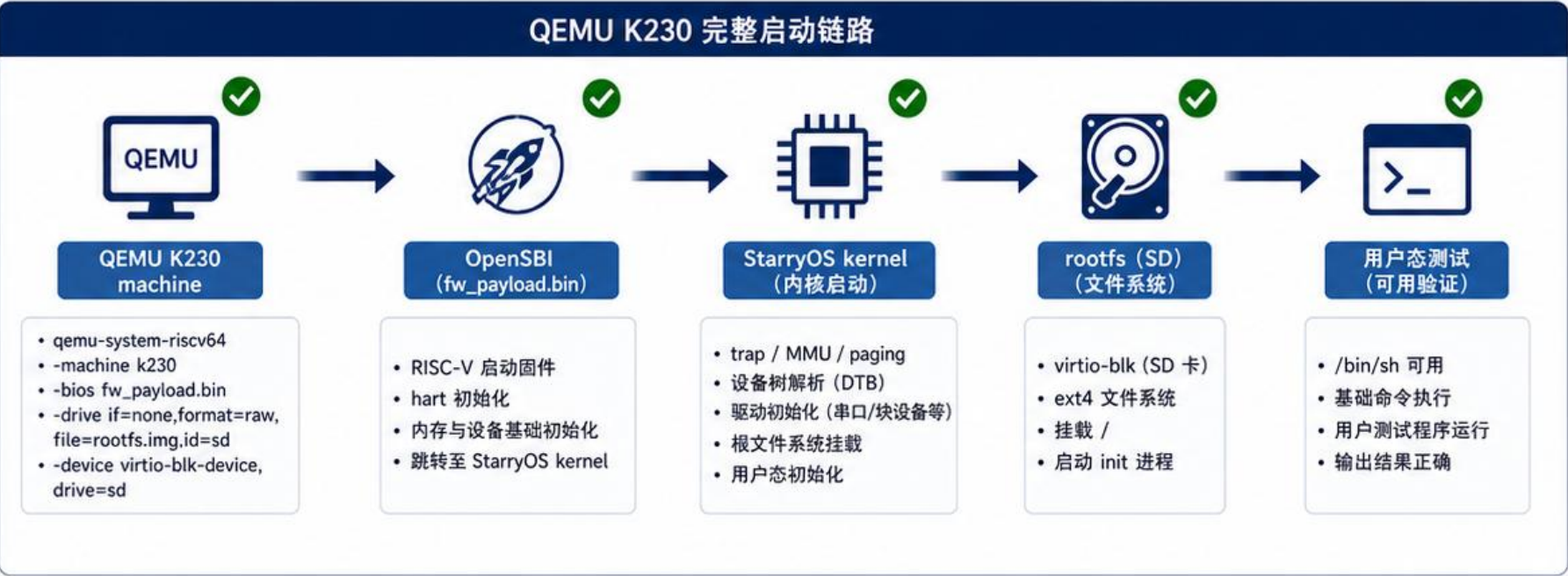
长期价值
进入 upstream/dev
长期维护路线

结论：迁移到 riscv64 动态平台是满足上游要求、降低工程债务、支撑后续 KPU 能力持续演进的**必选路径**。

第一步：让 StarryOS 在 QEMU K230 上完整启动

在进行 KPU/NPU 适配之前，先完成目标平台上“从固件到用户态”的完整启动链路，并能运行基础用户态测试。

- ✓ 新增 k230-canmv board 配置、DTS/DTB 和 qemu-k230.toml
- ✓ 接入 QEMU K230 machine、OpenSBI、StarryOS kernel
- ✓ SD rootfs 启动用户态测试
- ✓ 完整 boot/rootfs 是后续 KPU 验证前提



启动与用户态验证 (示例)

```
StarryOS (riscv64) #
kernel: booting ... [ OK ]
drivers: init ... [ OK ]
rootfs: mounted / (ext4) [ OK ]
init: starting ... [ OK ]

(init) login: root
# uname -a
StarryOS K230 riscv64
# ./test_kpu_stub
test: user space ready [ OK ]
#
```

✓ 状态：完整启动并可运行用户态测试程序
【Boot ✓ Rootfs ✓ User Test ✓】

准备证据

k230-canmv board 配置 (Board.toml)

设备树与 DTB (k230-canmv.dts / .dtb)

QEMU 配置文件 (qemu-k230.toml)

SD rootfs 镜像 (ext4 rootfs.img)

启动日志与用户态测试输出截图

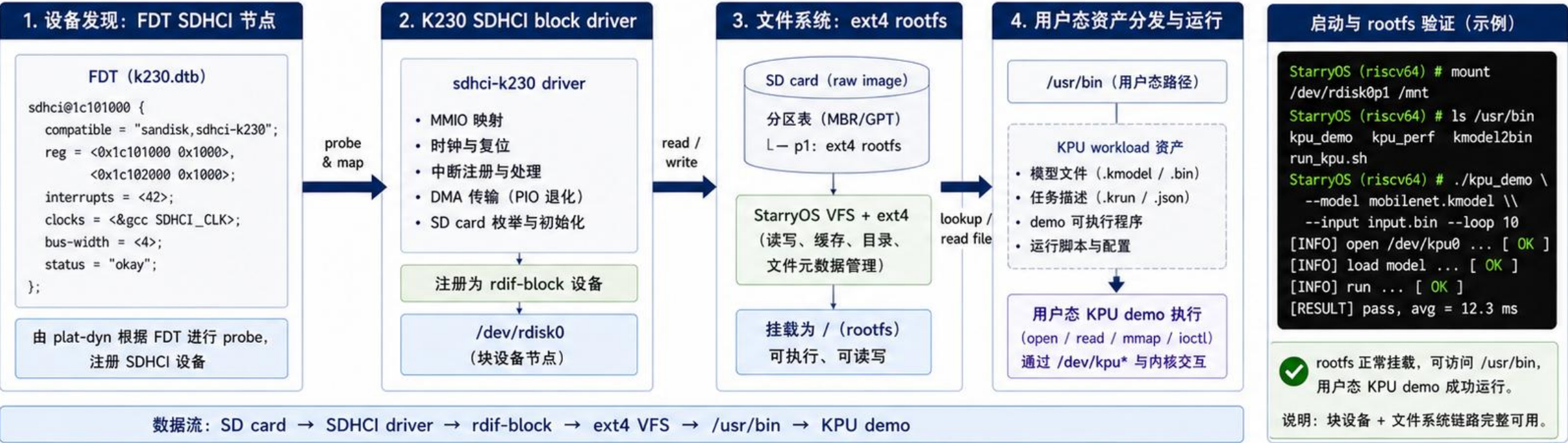
🎯 核心结论：StarryOS 已在 QEMU K230 上实现从固件到用户态的完整启动与可用，具备进行 KPU/NPU 验证的基础条件。

没有完整可用的 OS 启动链路，无法进行中断、内存、devfs 与 runtime 的端到端验证。

第二步：K230 SDHCI 作为 rootfs block device

• 为 StarryOS 提供稳定、可用的块设备与文件系统，承载模型与执行程序，支撑用户态 KPU workload 运行与验证。

- ✔ K230 QEMU 使用 SD raw image 提供 rootfs
- ✔ 新增 K230 SDHCI block driver
- ✔ 适配 rdif-block API
- ✔ KPU workload 以用户态资产分发到 /usr/bin



准备证据

FDT: sdhci@ 节点 (k230.dtb)

SDHCI 驱动初始化日志 / 中断统计

/dev/rdisk0 可用 lsblk / rdisk info

ext4 rootfs 挂载日志 mount / df -h

/usr/bin 列表与 KPU demo 执行成功截图

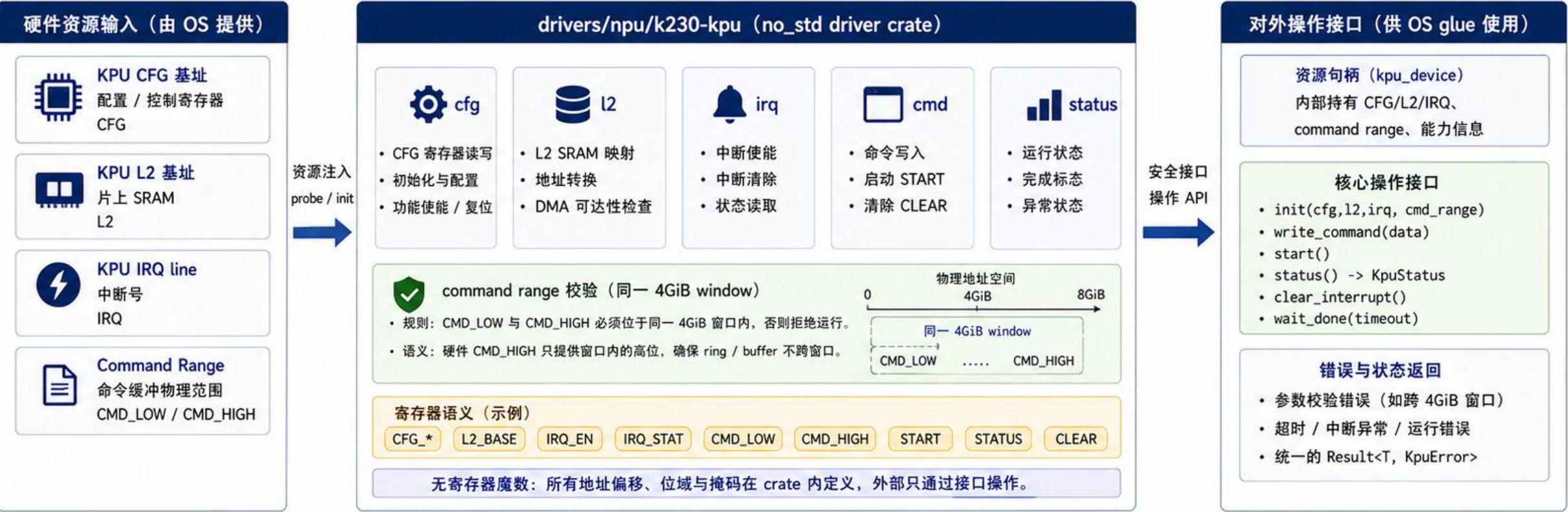
核心结论：K230 SDHCI block 设备驱动与 ext4 rootfs 已在 QEMU 上稳定可用，为用户态 KPU workload 提供可靠支撑。

没有 rootfs，就没有模型文件、.krun 与用户程序；SDHCI 是 StarryOS 支撑链上的关键一环。

第三步：KPU driver crate 与资源抽象

- 将 K230 KPU 硬件细节收敛在 **no_std driver crate** 中，通过清晰的资源与操作接口被 StarryOS glue 安全使用。

- drivers/npu/k230-kpu no_std driver crate
- 封装 CFG/L2/IRQ、command range、status、clear/start
- command range 校验同一 4GiB window
- 减少 StarryOS glue 中的寄存器魔数

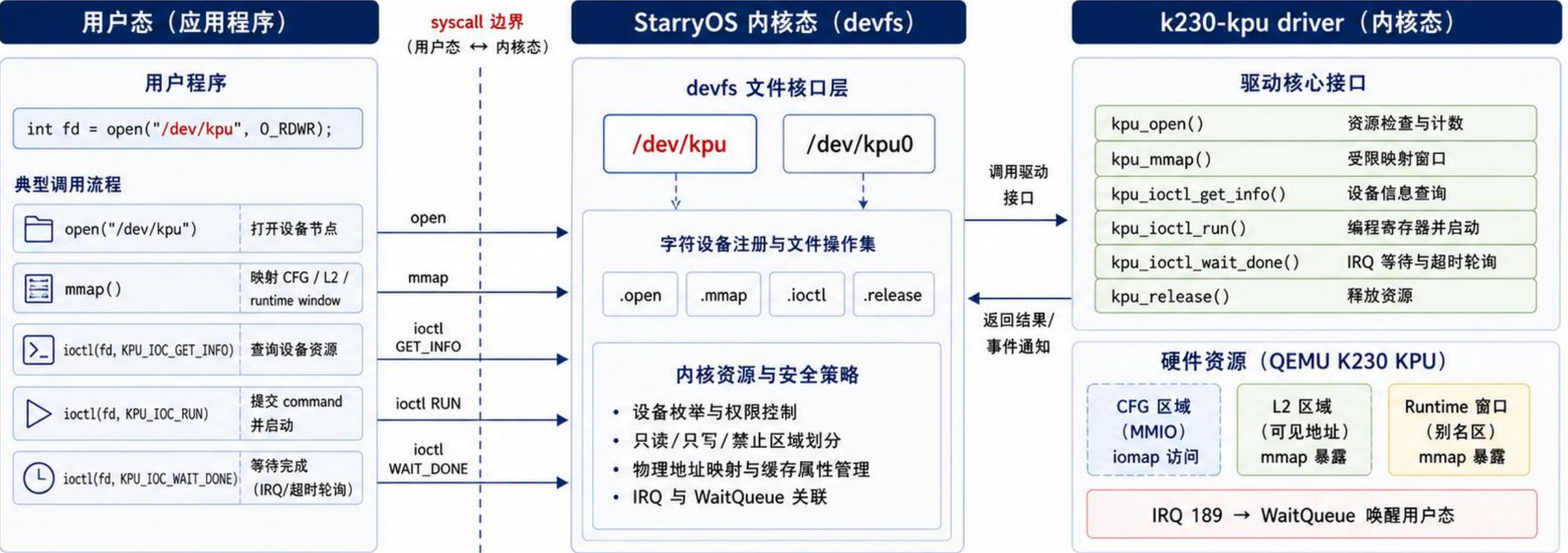


核心结论: 硬件细节被收敛在 driver crate, StarryOS 仅通过清晰资源与操作接口驱动 KPU。

同一 crate 完成资源管理、寄存器语义封装与参数校验, 让 KPU 驱动在 StarryOS 中更安全、更可维护。

第四步：StarryOS devfs 与 /dev/kpu

- `/dev/kpu` 和 `/dev/kpu0`
- `KPU_IOC_GET_INFO` 查询资源
- 受限 `mmap` 暴露 `CFG/L2/runtime window`
- `KPU_IOC_RUN + KPU_IOC_WAIT_DONE`

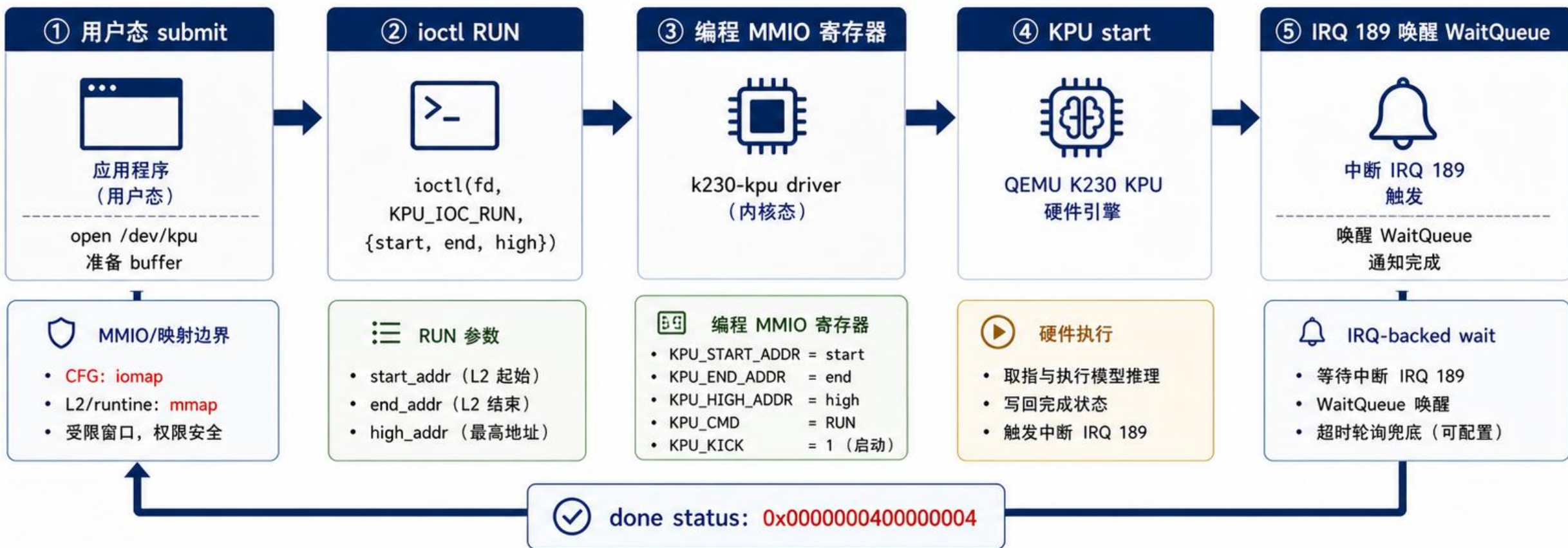


核心结论：通过 devfs + ioctl + 受限 mmap，把 KPU 安全地暴露给用户态程序。

用户态通过标准文件接口访问 `/dev/kpu`，不直接碰物理地址；内核侧统一资源、映射与中断策略，保障正确性与安全性。

第五步：MMIO、L2 mmap 与 IRQ-backed wait

- CFG 通过 iomap 访问，不当普通 RAM
- L2/runtime window 通过受限 mmap 暴露
- KPU_IOC_RUN 编程 start/end/high 并启动
- KPU_IOC_WAIT_DONE: IRQ + WaitQueue, 超时轮询兜底



核心结论：通过 MMIO + 受限 mmap + IRQ-backed wait, 形成安全、可同步、可演示的 KPU 执行路径。

一次 command submit 的核心时序：用户态 ioctl RUN → 编程 MMIO → KPU start → IRQ 189 → WaitQueue 唤醒 → 返回 done status。

第六步：从 smoke 到可检查输出

验证路线：从点亮设备 → 可检查 output

- open、寄存器读写、空 command 拒绝、done status
 - fake output window 验证 completion 写回 guest memory
- runtime arg table direct I/O 更接近真实 runtime
 - rootfs **.krun** 让 workload 可作为用户态资产分发

验证维度	① 设备可访问 (Smoke 点亮)	② 完成信号可见 (Done / IRQ)	③ 输出写回可见 (Fake Output)	④ 更真实的 runtime (Arg Table I/O)	⑤ 可分发的用户工作负载 (.krun)
 open / devfs 设备节点	✔ /dev/kpu 打开成功	✔ 持续通过	✔ 持续通过	✔ 持续通过	✔ 持续通过
 寄存器读写 KPU_IOC_*	✔ 寄存器读写正常	✔ 持续通过	✔ 持续通过	✔ 持续通过	✔ 持续通过
 空 command (Reject)	✔ 空跑 / 非法拒绝	✔ 持续通过	✔ 持续通过	✔ 持续通过	✔ 持续通过
 done status / IRQ	— 未验证	✔ done status 正常 IRQ 可达	✔ 持续通过	✔ 持续通过	✔ 持续通过
 output / memory 写回验证	— 未验证	— 未验证	✔ fake output window output hash changed KPU_SMOKE_PASS	✔ 持续通过	✔ 持续通过
 runtime arg table direct I/O	— 未验证	— 未验证	— 未验证	✔ arg table direct I/O 更接近真实 runtime	✔ 持续通过
 rootfs / .krun 可分发资产	— 未验证	— 未验证	— 未验证	— 未验证	✔ .krun 运行成功 可分发 / 可复现 .krun replay ready

 核心结论：验证链路已从 Smoke 覆盖完整性，逐步推进到可检查 output 和可分发资产。



验证方法逐步升级：先证明设备可访问，再证明完成信号，再证明 QEMU KPU 能把结果写回 StarryOS 管理的 buffer。

对标 kunOS: 完整 54 条 KPU command replay

- kunOS/K230 SDK RT-Smart
已跑过 YOLOv8n big-core runtime

- reference 捕获
完整 KPU workload

- 转换成 StarryOS
可复放的 .krun

- StarryOS 复放
54 条真实 KPU command

- 验证 run=54/54、IRQ
及每次运行的输出一致性



性质说明: 本阶段为 **reference capture** → **StarryOS replay**, 对齐的是底层 KPU workload; .kmodel 原生加载与调度将在后续原生 runtime 阶段进入 StarryOS。

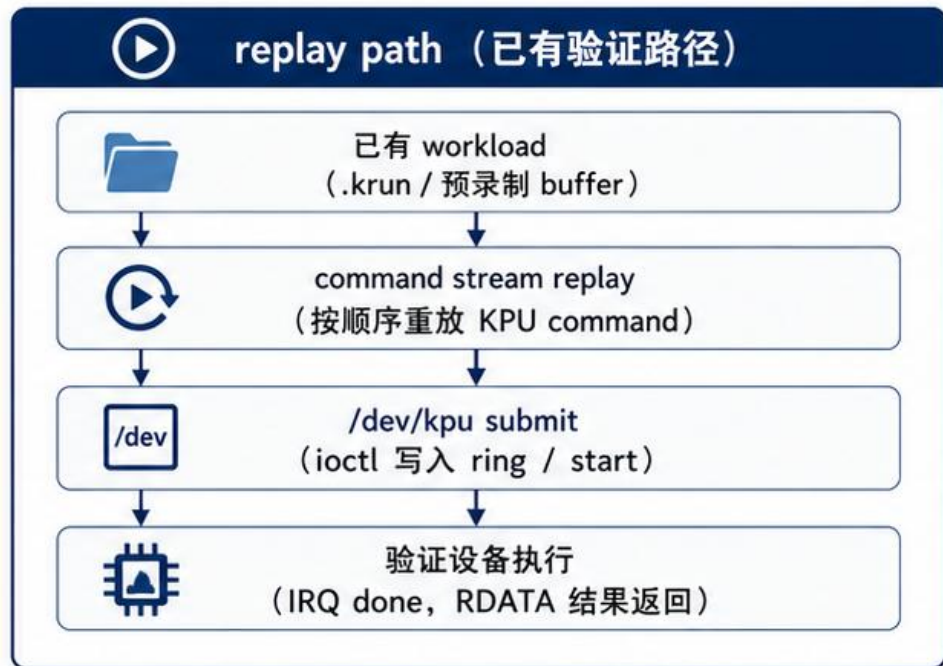


核心结论: 已完成 54 条 KPU command replay, 对齐 kunOS/RT-Smart YOLOv8n workload, 验证底层内核能力与时序正确性。

此为 replay/capture bridge 阶段, 不是原生 .kmodel 加载阶段; 原生 runtime 集成将在下一阶段推进。

从 replay 到原生 runtime: 目标进一步抬高

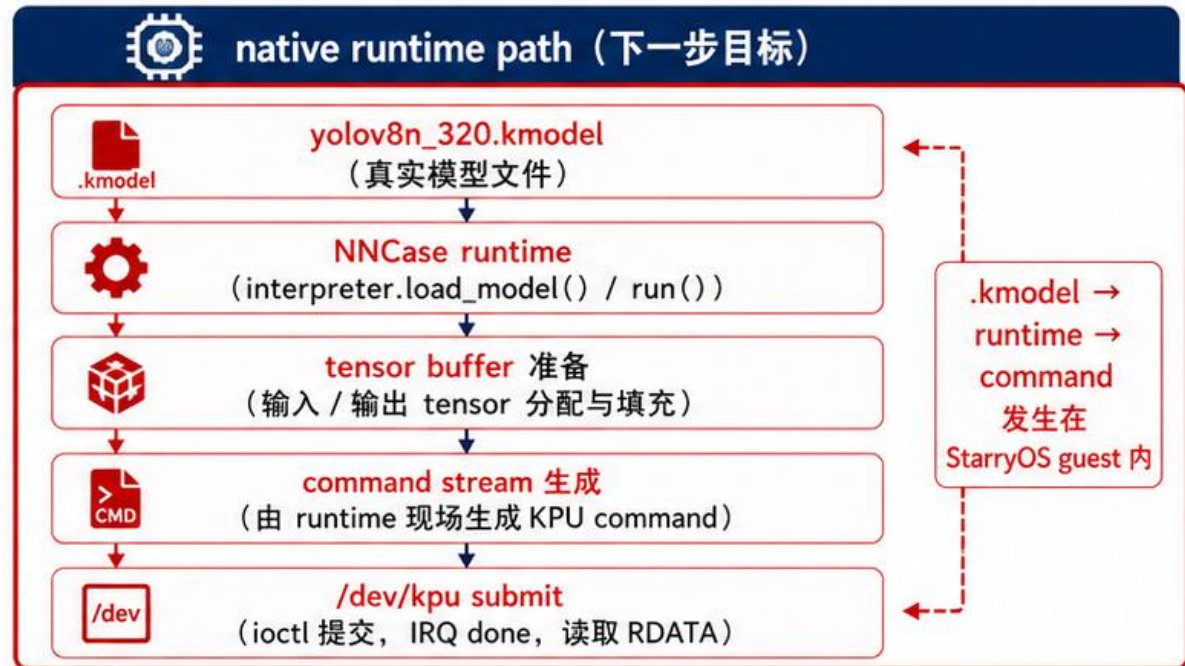
- 1 replay 证明 StarryOS 能执行真实 command
- 2 下一目标: **.kmodel** 解析、tensor 准备、command 生成发生在 **StarryOS guest 内**
- 3 复用官方 **NNCase runtime**, 降低格式逆向风险
- 4 工作目标从播放 workload 推进到**承载真实 runtime**



当前已达成

- 设备可访问、可映射、可提交、可中断、可读结果
- 证明 StarryOS + QEMU K230 KPU 能跑真实 command

目标抬高
从播放 workload
推进到承载现实 runtime



目标价值

- 支持真实 .kmodel 格式与官方 NNCase 生态
- 降低逆向与格式维护成本, 面向长期可信支持

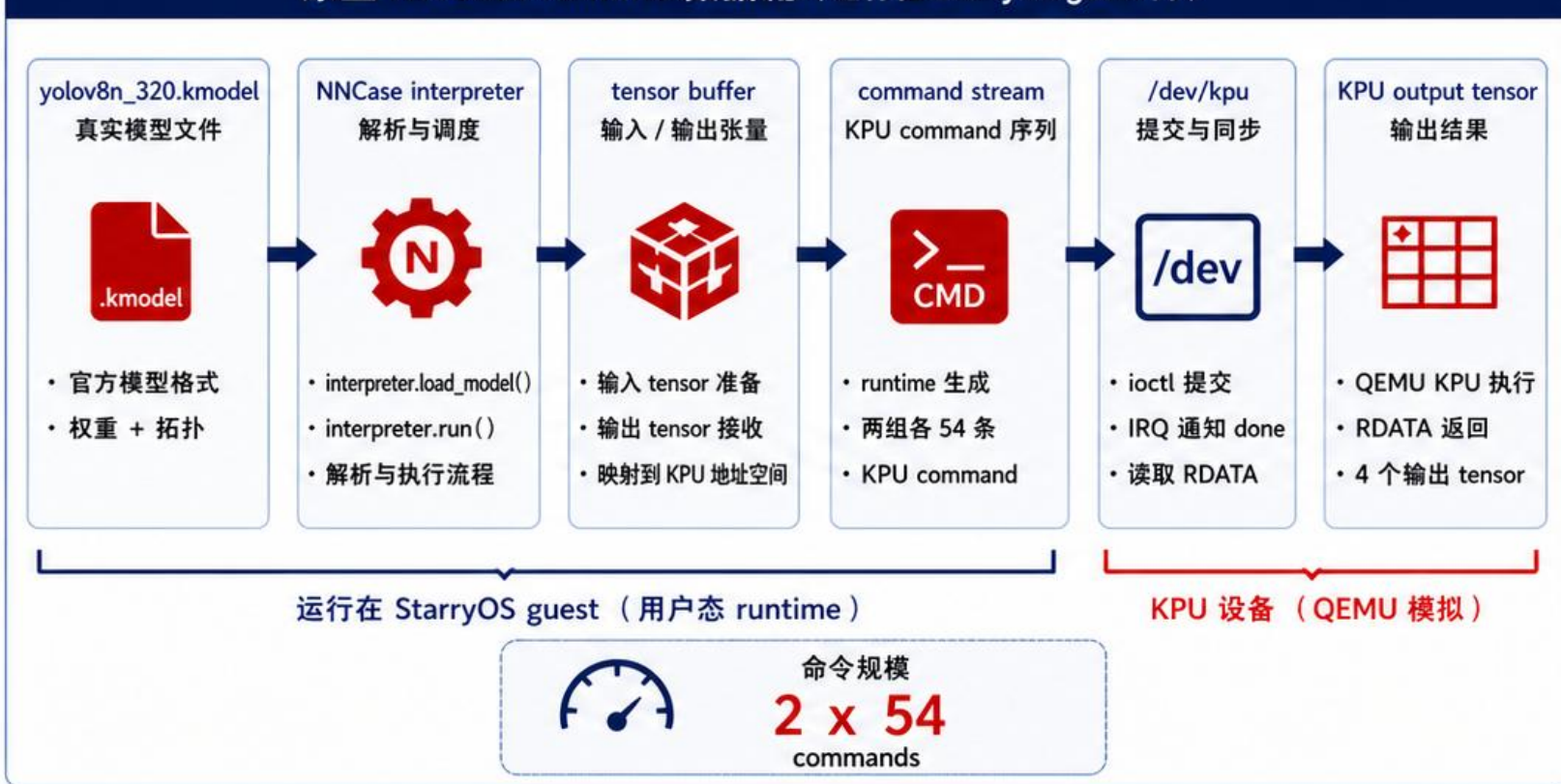


核心结论: 从设备可执行, 推进到 runtime 可承载。

原生 NNCase runtime: 真实 .kmodel 在 StarryOS 内运行

- 1 新增 kpu-nncase-runtime case
- 2 StarryOS guest 加载 yolov8n_320.kmodel
- 3 调用 interpreter.load_model() 和 interpreter.run()
- 4 runtime 现场生成两组各 54 条 KPU command
- 5 输出 NNCASE_MINIMAL_PASS、YOLOV8N_DEMO_PASS、K230_NNCASE_RUNTIME_PASS

原生 NNCase runtime 数据流 (运行在 StarryOS guest 内)



运行证据 (来自 kpu-nncase-runtime)

```
=== kpu-nncase-runtime case ===  
[INFO] load model: yolov8n_320.kmodel  
[INFO] interpreter.load_model() OK  
[INFO] interpreter.run() start  
[INFO] generate command stream ...  
[INFO] command group A: 54 commands  
[INFO] command group B: 54 commands  
[INFO] submit to /dev//kpu  
[INFO] wait IRQ done ..  
[INFO] read RDATA -> 4 output tensors
```

- ✓ [PASS] NNCASE_MINIMAL_PASS
- ✓ [PASS] YOLOV8N_DEMO_PASS
- ✓ [PASS] K230_NNCASE_RUNTIME_PASS

 2 x 54 commands

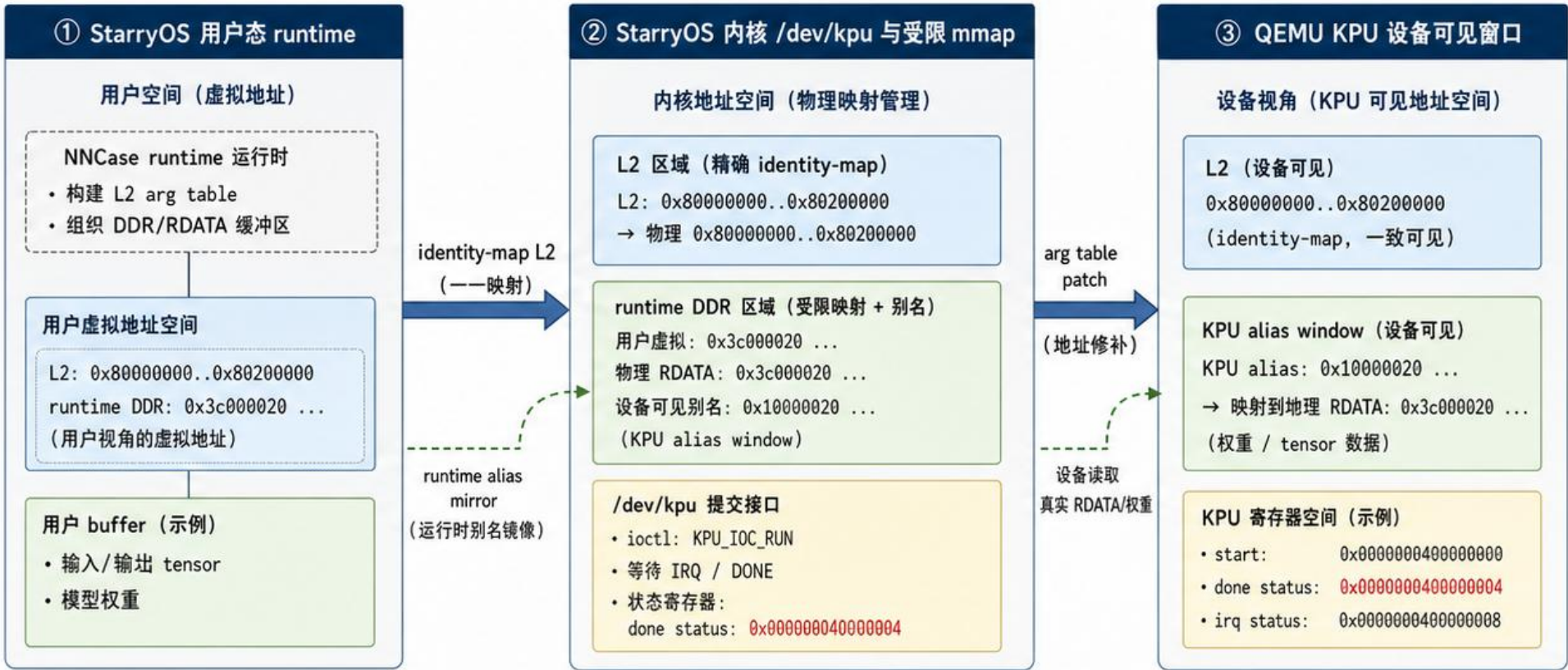
 RDATA returned: 4 output tensors

! 边界: YOLO 后处理语义仍需继续对齐。

最难的内核边界：地址窗口与 runtime 语义对齐

Kernel Addressing Boundary

- 官方 NNCase/K230 runtime 使用 L2 arg table 和 DDR/RDATA 地址组织 KPU submit ;
StarryOS 必须**精确处理**用户虚拟地址、物理地址、设备可见地址之间的关系。



关键结论

- 1 L2 必须 **identity-map**, 保证 runtime 写入 arg table。
- 2 低位 runtime window **不能粗暴 identity-map**, 以避免地址冲突与副作用。
- 3 通过 **alias mirror + patch**, 让 KPU 读取真实 RDATA/权重 数据。



核心结论：设备必须看到同一份真实运行时数据。

StarryOS 在 K230 KPU 适配中，构建了正确的地址语义与设备可见性，让官方 runtime 在 StarryOS 上安全、正确地运行。

当前支持程度与结论

A 底层设备支持完成



- K230 QEMU boot
- SD rootfs 挂载
- FDT probe 识别 KPU/NPU 设备
- KPU driver 初始化
- /dev/kpu 设备节点与 devfs 暴露
- mmap / ioctl 接口可用
- IRQ / done 中断与完成通知

证据（来自测试/运行结果）

- ✓ 设备节点 /dev/kpu 可访问
- ✓ MMIO 映射成功
- ✓ ioctl 提交 -> IRQ done -> 读取 RDATA
- ✓ QEMU KPU 正常执行 command
- ✓ 读回 RDATA 并返回

B 原生 runtime 承载完成



- runtime-style buffer 管理完善
- .krun replay 能够稳定执行
- 真实 .kmodel 加载成功
- NNCase interpreter 解析并运行
- NNCase 生成 54 条 KPU command

证据（来自测试/运行结果）

- ✓ interpreter.load_model() OK
- ✓ interpreter.run() start
- ✓ generate command stream
- ✓ command group A/B: 54 commands

C 结果读取完成



- QEMU KPU 读取权重 / RDATA
- 输出 4 个非零 tensor
- 每个 tensor 均有 hash / stats

证据（来自测试/运行结果）

- ✓ read RDATA -> 4 output tensors
- ✓ 所有输出 tensor 为非零
- ✓ 记录 tensor hash 与 abs-sum stats

边界 / 下一步

1



已提交

- 面向 upstream/dev 的 draft PR #1036

draft PR #1036 -> upstream/dev

2



剩余

- YOLO 检测框语义和官方 RT-Smart reference 后处理继续对齐

3



边界（当前不保证）

- YOLO 检测结果与官方完全一致
- 模型语义/后处理细节仍需对齐



结论：StarryOS 已具备 K230 KPU 底层设备支持与原生 runtime 承载能力。



Demo展示



章节

chapter

总结与展望



AI时代，人类走向何方？

MLsys: AI先攻破ML，还是先攻破
Sys?

AI时代的学习与教育会变成什么样？





几点个人感受：

知识常用常新，不用则遗忘很快

学习知识的作用很大程度上是培养一种“直觉”，学过的东西在再需要使用的時候捡起来更快。

鼓励交叉，鼓励尝试，人脑或许也有“涌现”？

AI时代，创新能力，质疑能力尤为可贵。

AI 时代最重要的变化，是答案变得廉价，判断变得昂贵。
代码也会变得廉价，证据会变得昂贵。

AI时代，让教育的“个性化”成为可能



AI时代的OS课程

操作系统不是通过逐行审查每一个用户程序来保证系统可靠运行的。

OS 的方法是：默认用户程序不可信，然后通过进程隔离、地址空间、权限检查、系统调用边界、资源配额、异常处理等机制，把不可信代码限制在可控范围内。

AI 生成代码也是类似的。我们不能假设人能完全审查所有生成代码，而应该训练学生设计边界、接口、约束、测试和观测机制，让不可信实现必须在系统规则内运行。

正因为 AI 生成内容不可完全人工审查，系统思维才更重要。

我们应该从OS课程学到的：
面对复杂、不完全可信、不可完全穷尽理解的系统，如何通过抽象、隔离、约束、观测和验证来获得可控性。

代码检查员



系统责任人



关于课程设计的一点建议

助教人数少，学生单打独斗，交流偏少，引入AI 助教？

AI赋能后，大家都可以上手高复杂度的内容，或许课程可以更为自由？但资源有限是限制因素。

可以有更多open project？学生可以借助 AI 生成代码，但要自己理解系统，自己定义正确性，自己承担判断责任，能够评价系统是否正确、可靠、高效、安全。

AI 时代，人类的核心能力会从“生产答案”转向“定义问题和评价答案”。





希望OS Biglab能被更多同学所拥抱！

祝愿OS Biglab越办越好！

